

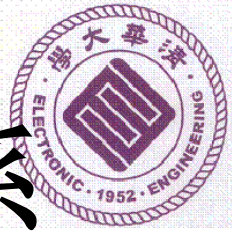
清华大学电子工程系

电子信息科学与技术导引

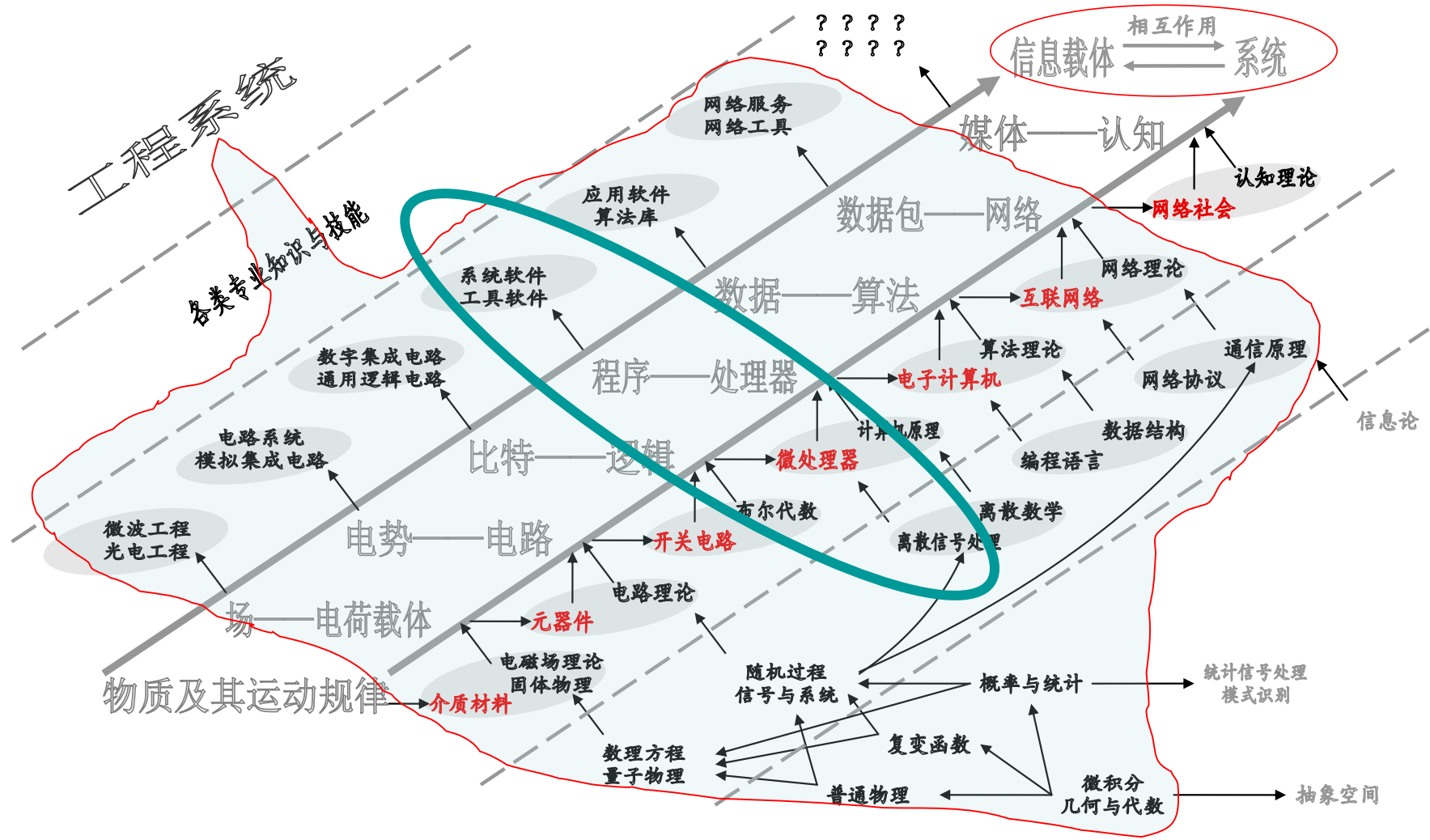


程序与处理器

马洪兵



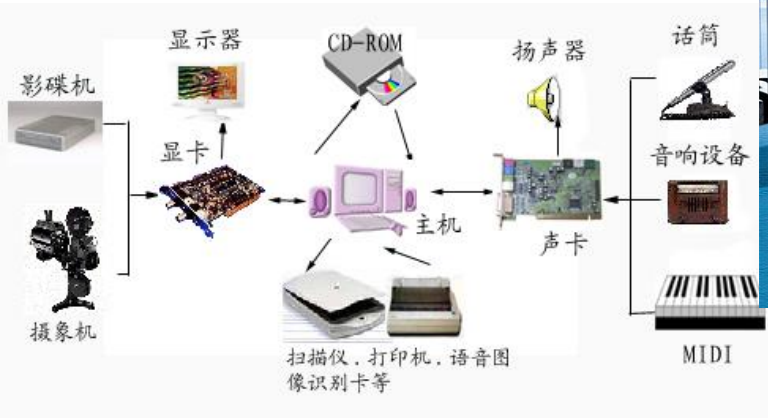
电子信息科学与技术知识体系



电子信息科学与技术知识体系

“程序与处理器”定位：
联系硬件与软件的桥梁

硬件

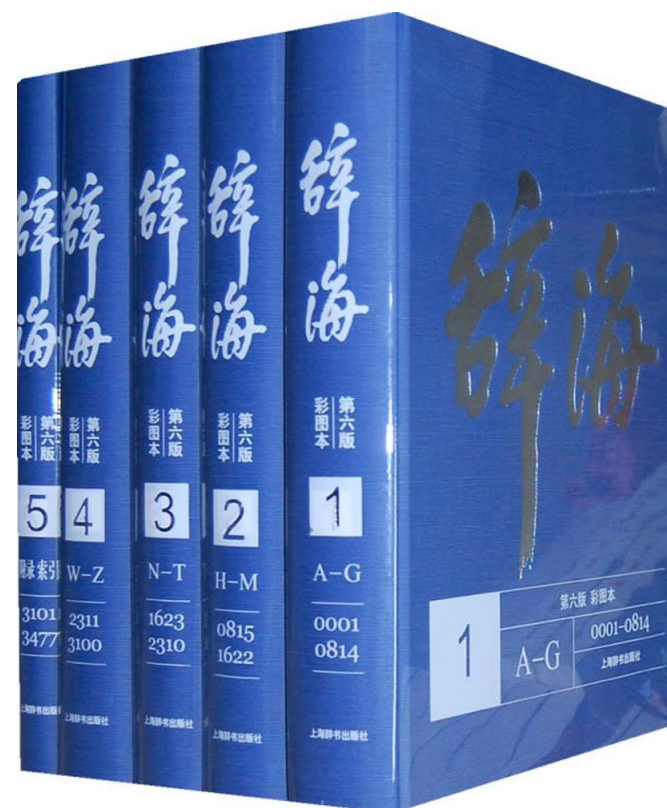


软件



什么是程序？

- ① 行事的先后次序；有序的步骤。如：工作程序；司法程序；医疗程序。
- ② 为使电子计算机执行一个或多个操作，或执行某一任务，按序设计的计算机**指令**的集合。



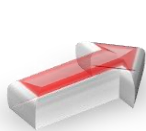


什么是处理器？

- 计算机系统中能独立执行程序，完成对数据和指令进行加工与处理的部分。是计算机系统的核心设备。由数据处理部件（运算器）、指令处理部件（指令部件）以及存储控制器组成。用于控制和执行计算机基本**指令系统**的处理器，称“中央处理器”。



大纲



1

从算盘到ENIAC

2

通用计算机模型

3

指令集体系结构

4

程序和程序设计语言

5

处理器的工作原理

6

性能问题

7

小结

人类早期的计算工具

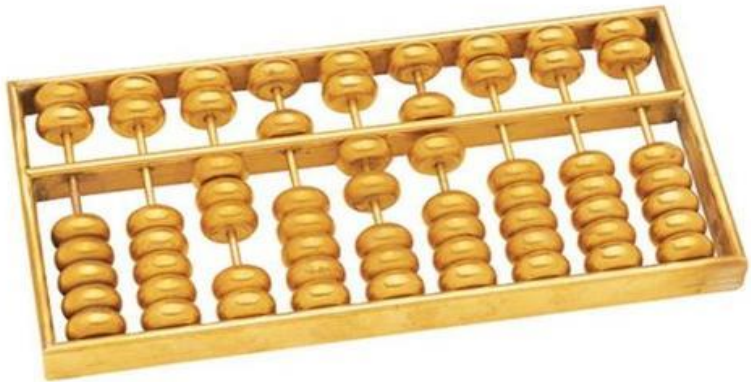


• 处理器——算盘

- 其形长方，周为木框，内贯直柱，俗称“档”。一般从九档至十五档，档中横以梁，梁上两珠，每珠作数五，梁下五珠，每珠作数一，运算时定位后拨珠计算，可以做加减乘除等算法

人类早期的计算工具

- 程序——储存在人的大脑中



加法口诀表

不进位的加			进位的加	
加数	直加	满五加	进十加	破五进十加
一	一上一	一下五去四	一去九进一	
二	二上二	二下五去三	二去八进一	
三	三上三	三下五去二	三去七进一	
四	四上四	四下五去一	四去六进一	
五	五上五		五去五进一	
六	六上六		六去四进一	六上一去五进一
七	七上七		七去三进一	七上二去五进一
八	八上八		八去二进一	八上三去五进一
九	九上九		九去一进一	九上四去五进一

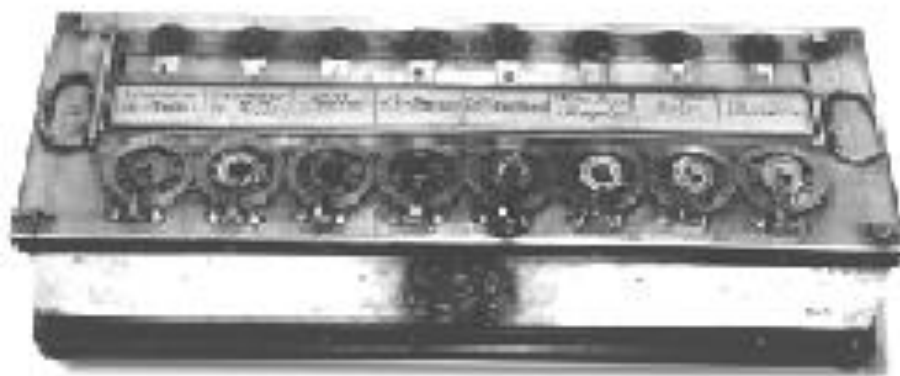
减法口诀表

不退位的减			退位的减	
减数	直减	破五减	退位减	退十补五的减
一	一下一	一上四去五	一退一还九	
二	二下二	二上三去五	二退一还八	
三	三下三	三上二去五	三退一还七	
四	四下四	四上一去五	四退一还六	
五	五下五		五退一还五	
六	六下六		六退一还四	六退一还五去一
七	七下七		七退一还三	七退一还五去二
八	八下八		八退一还二	八退一还五去三
九	九下九		九退一还一	九退一还五去四

人类早期的计算工具



Blaise Pascal(1623-1662)

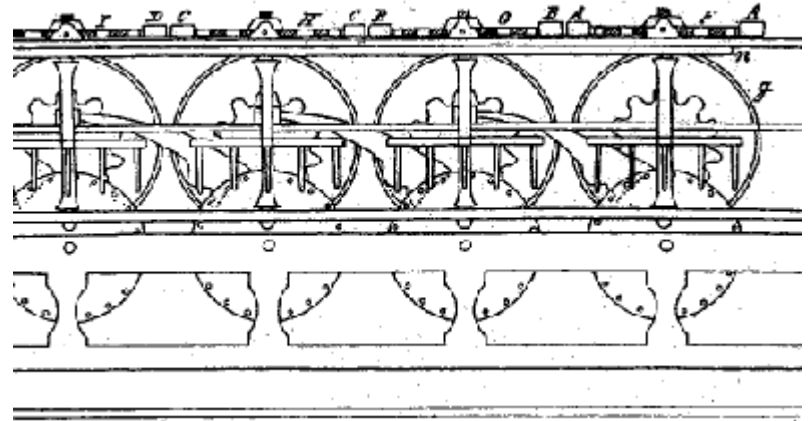
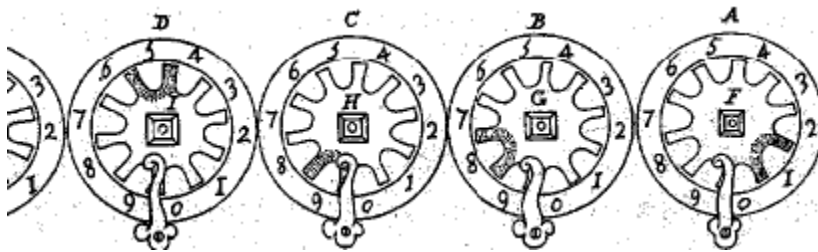


加法机 (1642)

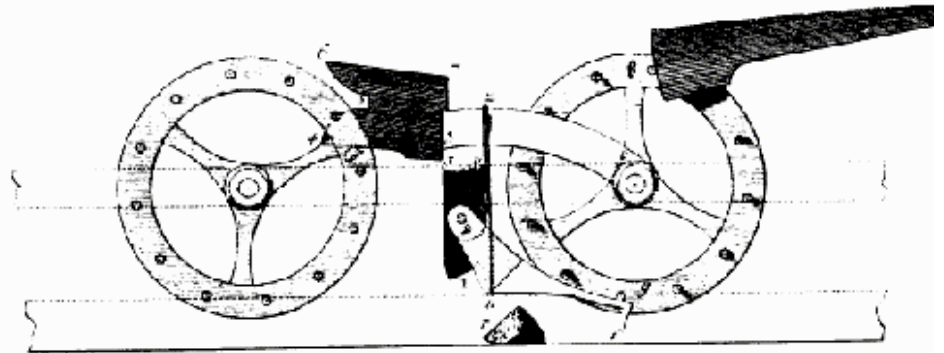
- 由一系列齿轮组成的纯机械设备。采用十进制运算，能完成8位十进制数的加法和减法运算
- 计算程序体现在设备的机械结构中
- 意义：揭示出用机器代替人类进行计算，完全可以做到

人类早期的计算工具

Pascal加法机的工作原理

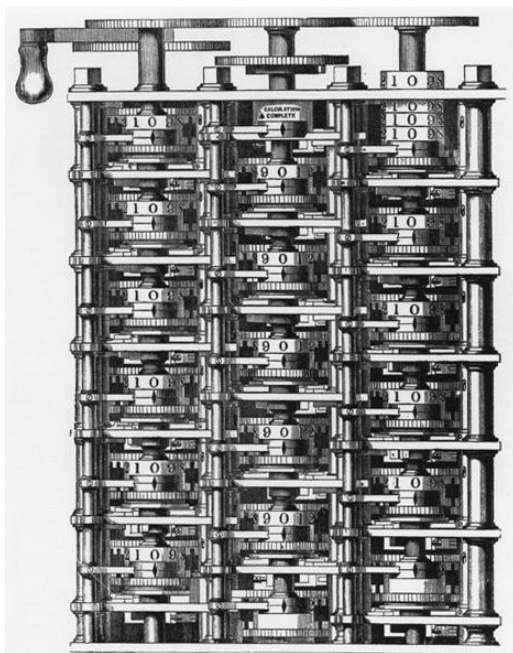


- **进位问题：**采用了一种小爪子式的棘轮装置，当个位齿轮朝**9**转动时，棘爪便逐渐升高，一旦齿轮转到**0**，棘爪就跌落下来，推动十位数的齿轮前进一档



人类早期的计算工具

- **Pascal**加法机发明后的300年间，出现了**Leibniz**计算器、**Babbage**分析机、**Hollerith**制表机等重要的机械计算机，但是发展速度十分缓慢



ENIAC

- 1946年, ENIAC在美国宾夕法尼亚大学研制成功, 这是世界上第一台现代意义的通用电子计算机



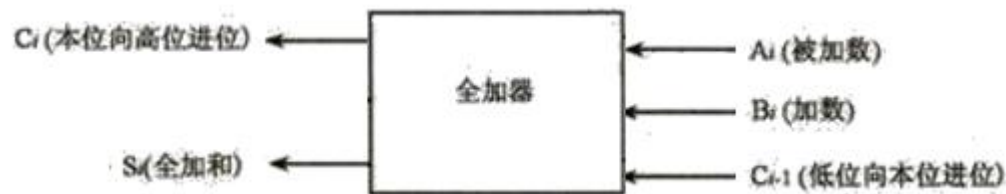
ENIAC

- 装有**18,000**个电子管，
总重量达**30**吨，功耗
140kw
- 有**20**个寄存器，每个能
存放**10**位的十进制数，
运算速度为每秒**5000**次
加法
- 通过设置分布在各处的
6000个开关和众多的插
头与插座来编程



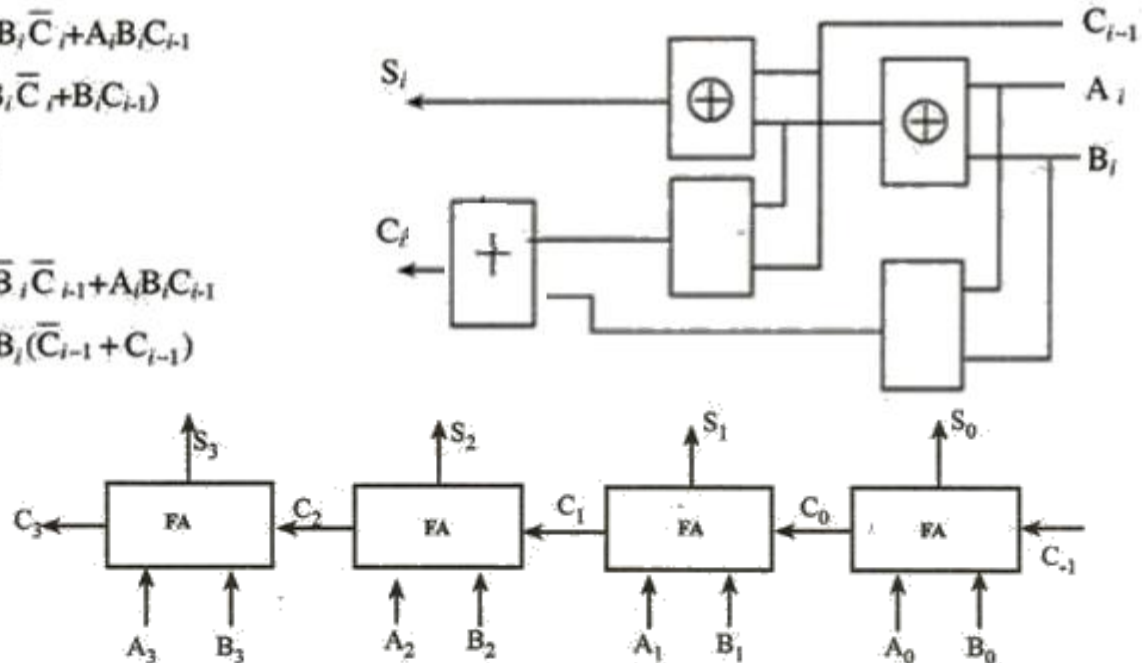
电子计算机如何实现计算？

- 作为计算基础的二进制加法



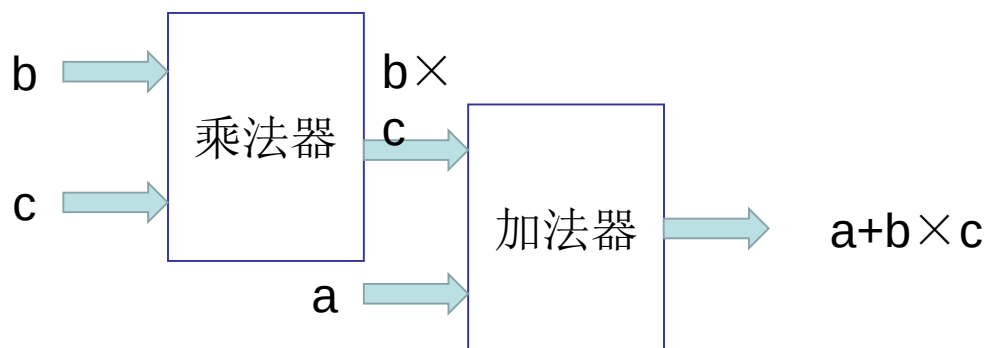
$$\begin{aligned}
 S_i &= \bar{A}_i \bar{B}_i C_{i-1} + \bar{A}_i B_i \bar{C}_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i B_i C_{i-1} \\
 &= \bar{A}_i (\bar{B}_i C_{i-1} + B_i \bar{C}_{i-1}) + A_i (B_i \bar{C}_{i-1} + \bar{B}_i C_{i-1}) \\
 &= \bar{A}_i (B_i \oplus C_{i-1}) + A_i \overline{(B_i \oplus C_{i-1})} \\
 &= A_i \oplus B_i \oplus C_{i-1}
 \end{aligned}$$

$$\begin{aligned}
 C_i &= \bar{A}_i \bar{B}_i C_{i-1} + A_i \bar{B}_i C_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i B_i C_{i-1} \\
 &= C_{i-1} (A_i \bar{B}_i + A_i B_i) + A_i B_i (\bar{C}_{i-1} + C_{i-1}) \\
 &= C_{i-1} (A_i \oplus B_i) + A_i B_i
 \end{aligned}$$



更复杂的计算

- 假如我们要计算： $a+b \times c$



- 如果我们要算另外一道题： $a \div b - c \times d$?

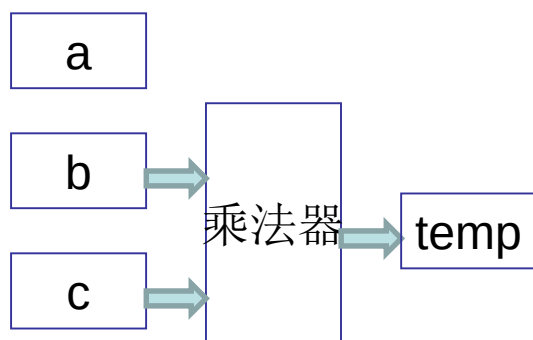


ENIAC的工作原理

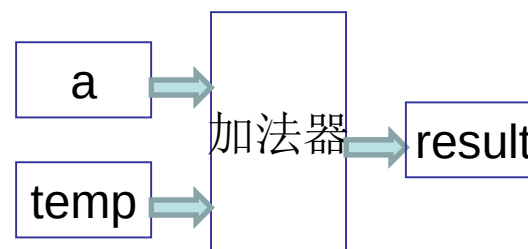
- 换一种思路：人类天生具备大局观，最擅长从大到小，由表及里的分析问题
- 计算也是如此——任何复杂的计算都是由简单的基本计算组合而成的

ENIAC的工作原理

- 我们用两个步骤来计算 $a+b \times c$:
 - 第一步: $\text{Temp} = b \times c$
 - 第二步: $\text{Result} = a + \text{temp}$
- 于是我们就可以:



第一步



第二步

- 当我们需要算另外一道题时，只要将这个过程进行修改即可

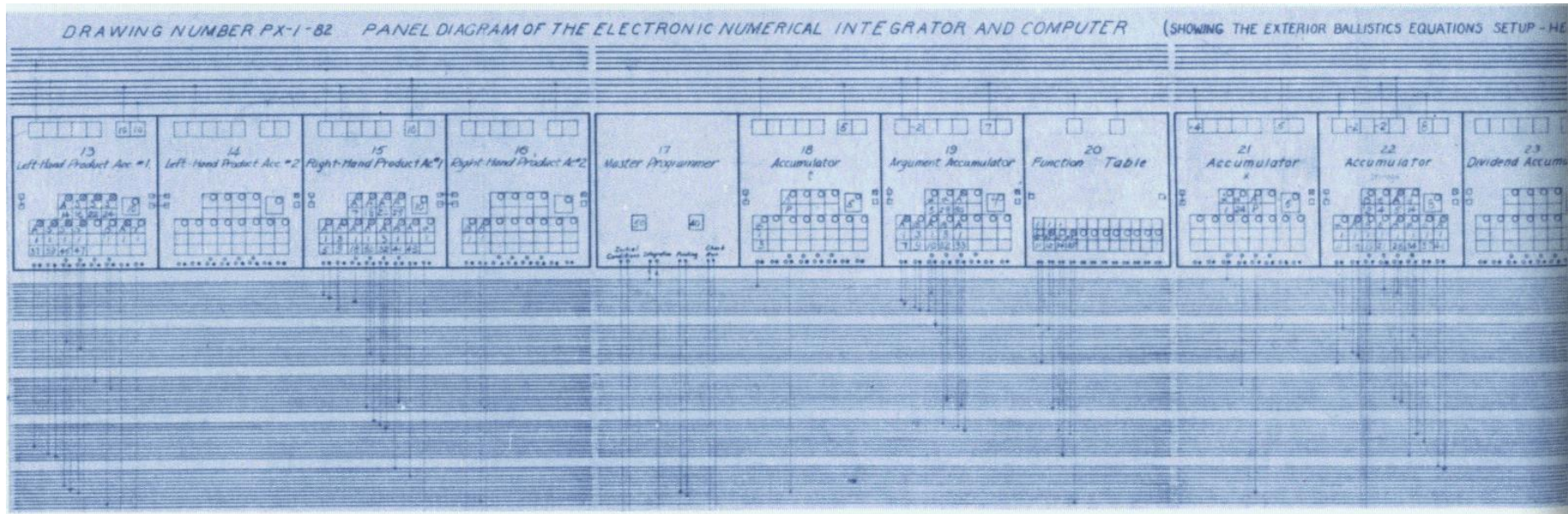


ENIAC的工作原理

- 基本计算→指令
- 基本计算的集合→指令集
- 基本运算的数量是有限的→指令集是有限集合，所以指令集中的指令（基本计算）可以通过电路来实现
- 将指令集中的指令（实现基本计算的电路）按某种方式进行排列组合，就可以完成不同的计算任务
- 程序→解决某个特定的问题的有限指令序列

ENIAC的工作原理

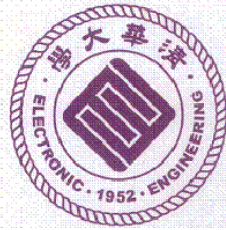
- ENIAC是一种通用计算机
- 为ENIAC编写的“程序”





大纲





图灵机



Alan Turing(1912-1954)

- **1936年4月**，即将要从剑桥大学国王学院毕业的图灵，发表了论文《论可计算数及其在判定问题中的应用(On Computable Numbers, With an Application to the Entscheidungsproblem)》，在这篇论文中他设计了被后人称为**图灵机**的计算机模型，这一年他**24岁**

230

A. M. TURING

[Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO
THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable numbers, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbersome technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.

Although the class of computable numbers is so great, and in many ways similar to the class of real numbers, it is nevertheless enumerable. In § 8 I examine certain arguments which would seem to prove the contrary. By the correct application of one of these arguments, conclusions are reached which are superficially similar to those of Gödel†. These results

† Gödel, “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I”, *Monatsh. Math. Phys.*, 38 (1931), 173–198.

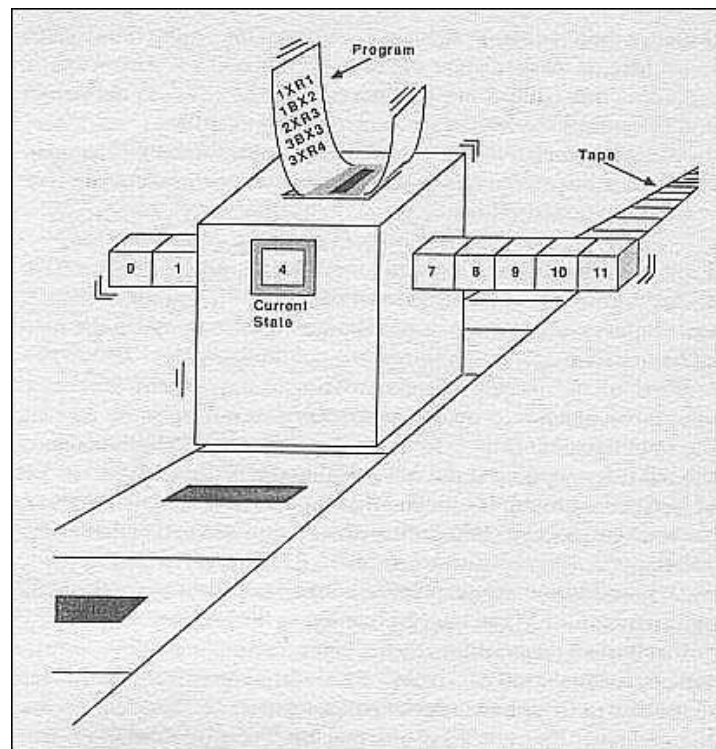


图灵机

- 图灵的基本思想是用机器来模拟人们用纸笔进行数学运算的过程，他把这样的过程看作下列两种简单的动作：
 - 在纸上写上或擦除某个符号
 - 把注意力从纸的一个位置移动到另一个位置
 - 而在每个阶段，人要决定下一步的动作，依赖于 **(a)** 此人当前所关注的纸上某个位置的符号和 **(b)** 此人当前思维的状态

图灵机

- 为了模拟人的这种运算过程，图灵构造出一台假想的机器，该机器由以下几个部分组成：
 - 一条无限长的纸带 **TAPE**
 - 一个读写头 **HEAD**
 - 一套控制规则 **TABLE**
 - 一个状态寄存器
- 图灵证明了这样的一台机器能够模拟人类所能进行的任何计算过程





图灵机

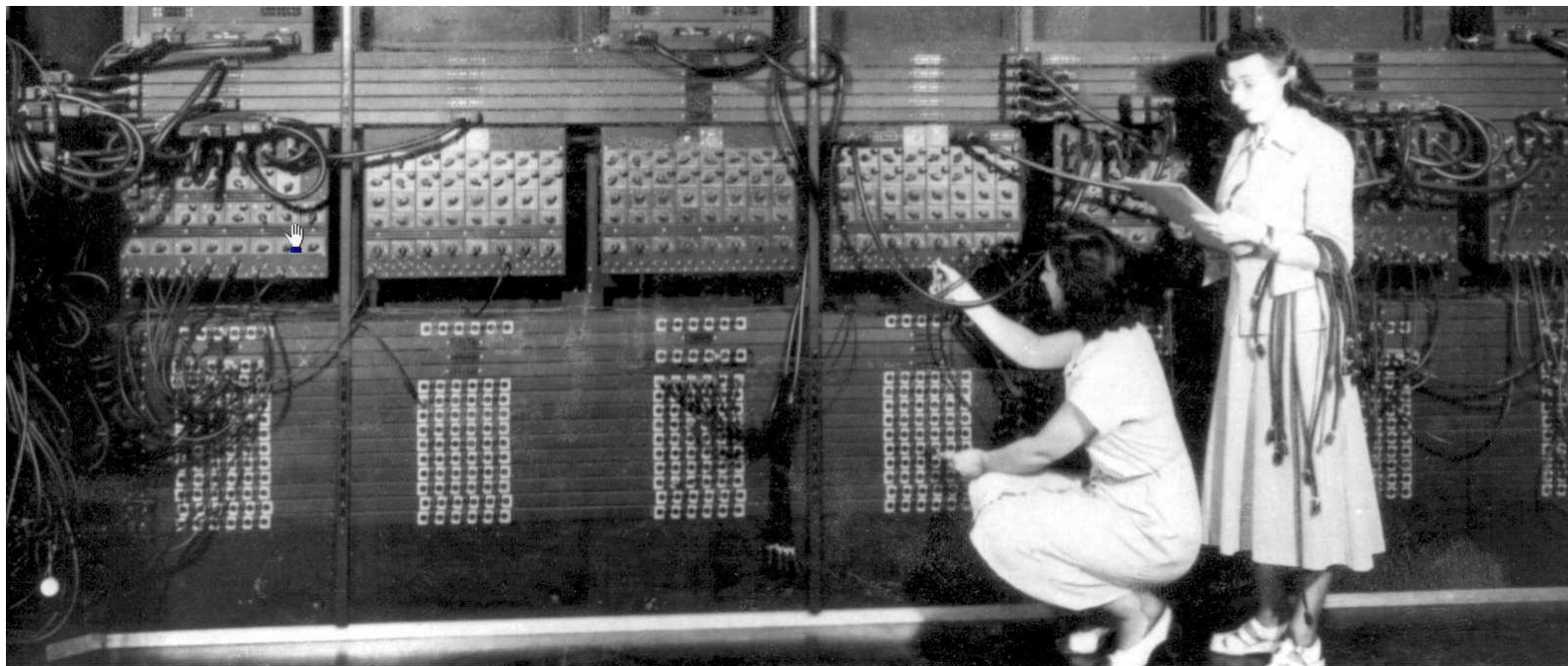
- 除了图灵机以外，人们还发明了很多其它的计算模型
 - 寄存器机
 - 递归函数
 - λ 演算
- 可以证明，这些模型无一例外地都和图灵机等价



ENIAC与图灵机的等价性

- 理论上可以证明，**ENIAC**与图灵机是等价的，因此**ENIAC**能够完成人类所能进行的任何计算过程
- 严格地说，**ENIAC**不是一台图灵机，而是若干台图灵机
- **ENIAC**的每一个电路结构，都是一个不同的图灵机

ENIAC的问题



- ❖ **ENIAC**是通用计算机
- ❖ 但是，为了让**ENIAC**完成不同的任务，工作人员必须改变它的电路结构

ENIAC的问题

- 冯·诺伊曼在**ENIAC**的研制过程中，就发现了**ENIAC**存在的两个主要问题：
 - 用大量的开关和插头来编程
 - 用**10**个电子管表示一位十进制数

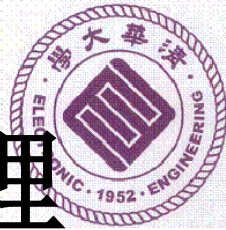


Von Neumann
(1903-1957)



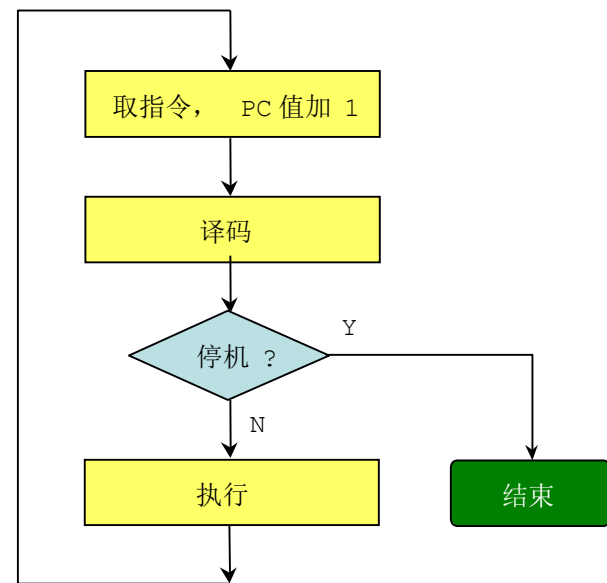
冯·诺依曼存储程序原理

- 冯·诺依曼提出的改进措施：
 - 针对用**10**个电子管表示一位十进制数，提出应该用二进制来替代
 - 针对**ENIAC**用开关和插头来编程，提出了程序可以以编码形式和数据一起在内存中存储
- 核心思想：指令（实现基本计算的电路）用二进制来编码
- 冯·诺伊曼的工作成为如今广泛采用的**冯·诺伊曼体系结构**的基础

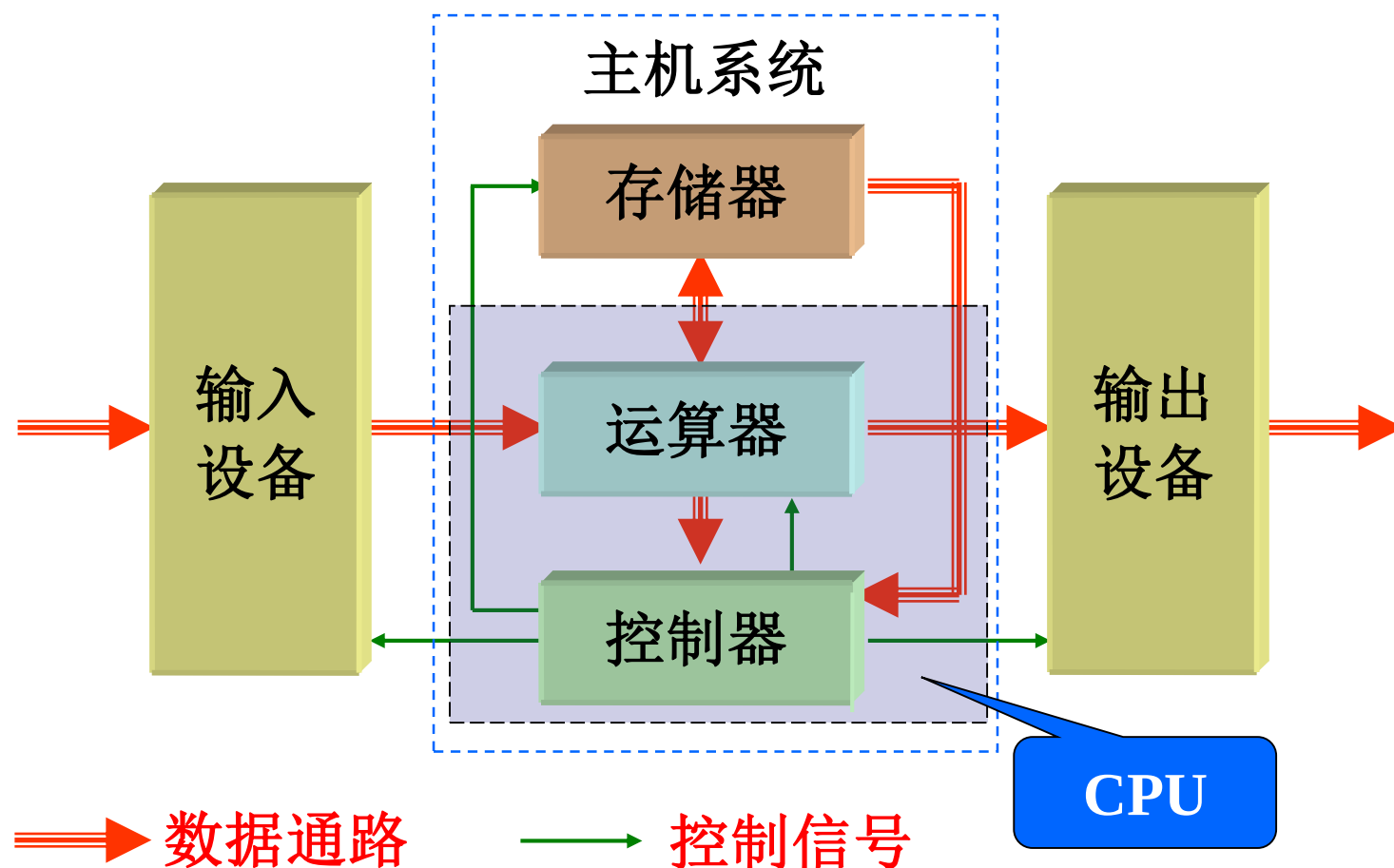


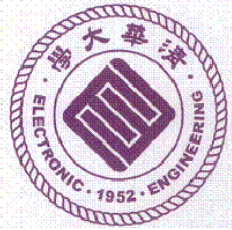
冯·诺伊曼机中程序的执行原理

- 把程序(指令序列)存储在存储器内,使计算机能像快速存取数据一样地快速存取组成程序的指令,存放位置由地址指定,地址码也是二进制形式
- 控制器根据存放在存储器中的程序工作,并由一个程序计数器(PC)控制指令的读取。控制器具有判断能力,能够根据计算结果选择不同的动作流程
- 给出程序中第一条指令的地址,控制器就可依据存储程序中的指令顺序周而复始地取指令、译码、执行,直到完成全部指令操作为止



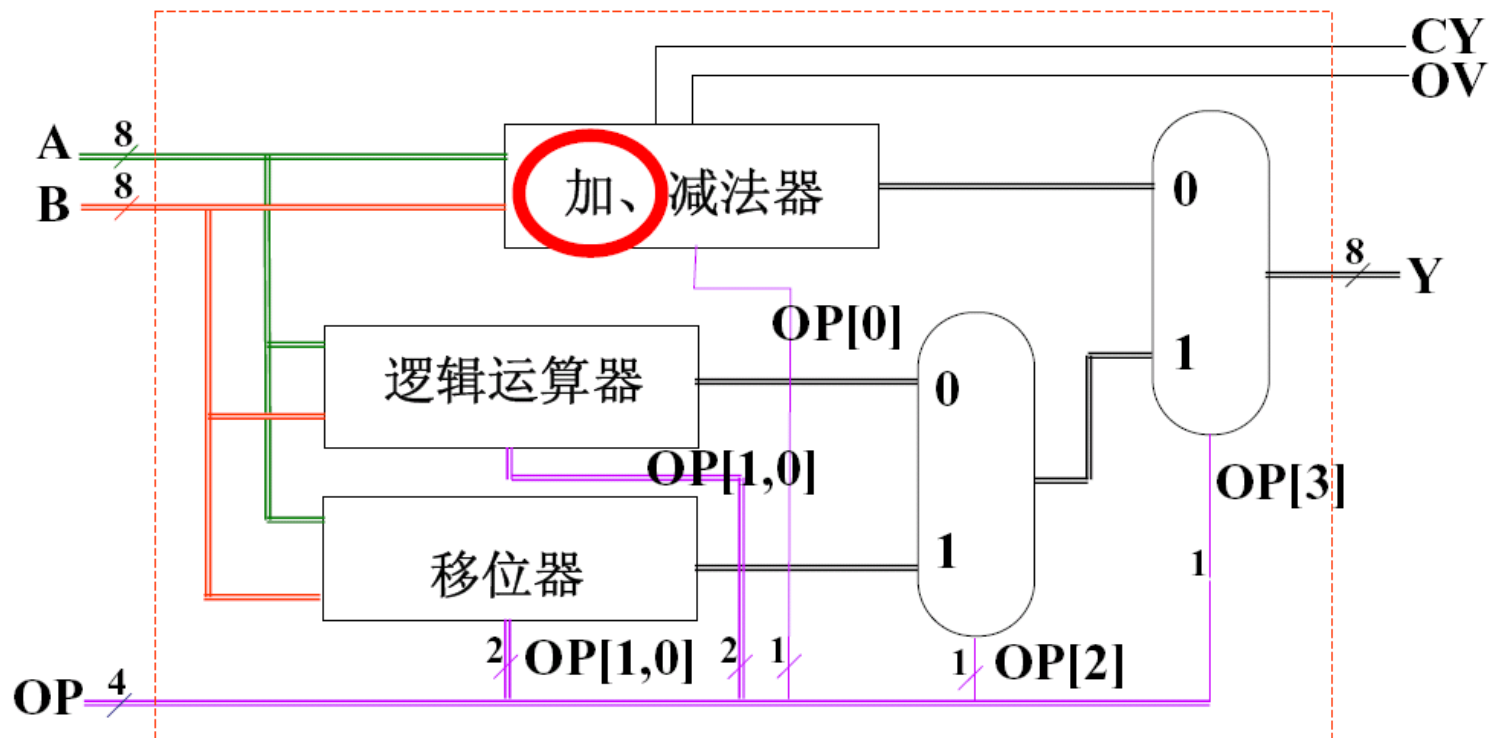
冯·诺依曼计算机组成





运算器——算术逻辑单元

- 实现基本计算的电路（如加法器、乘法器等）可以组合在一起，称为算术逻辑单元
ALU



存储器

- 复杂的计算需要大量的输入数据和输出数据，因此我们需要一个存储器将其存起来





控制器

- 除了存储器和运算器，还需要一个控制器
- 控制器控制着将存储器中的数据送到运算器(ALU)中进行计算，然后将结果存回到存储器中
- 指示输入数据存在何处，做什么运算，结果存到何处的控制命令就是指令

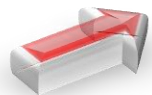
输入设备&输出设备

- 数据要从外部输入进来，结果也要输出出去，因此还需要输入设备和输出设备





大纲

- 
- 1 从算盘到ENIAC
 - 2 通用计算机模型
 - 3 指令集体系结构
 - 4 程序和程序设计语言
 - 5 处理器的工作原理
 - 6 性能问题
 - 7 小结

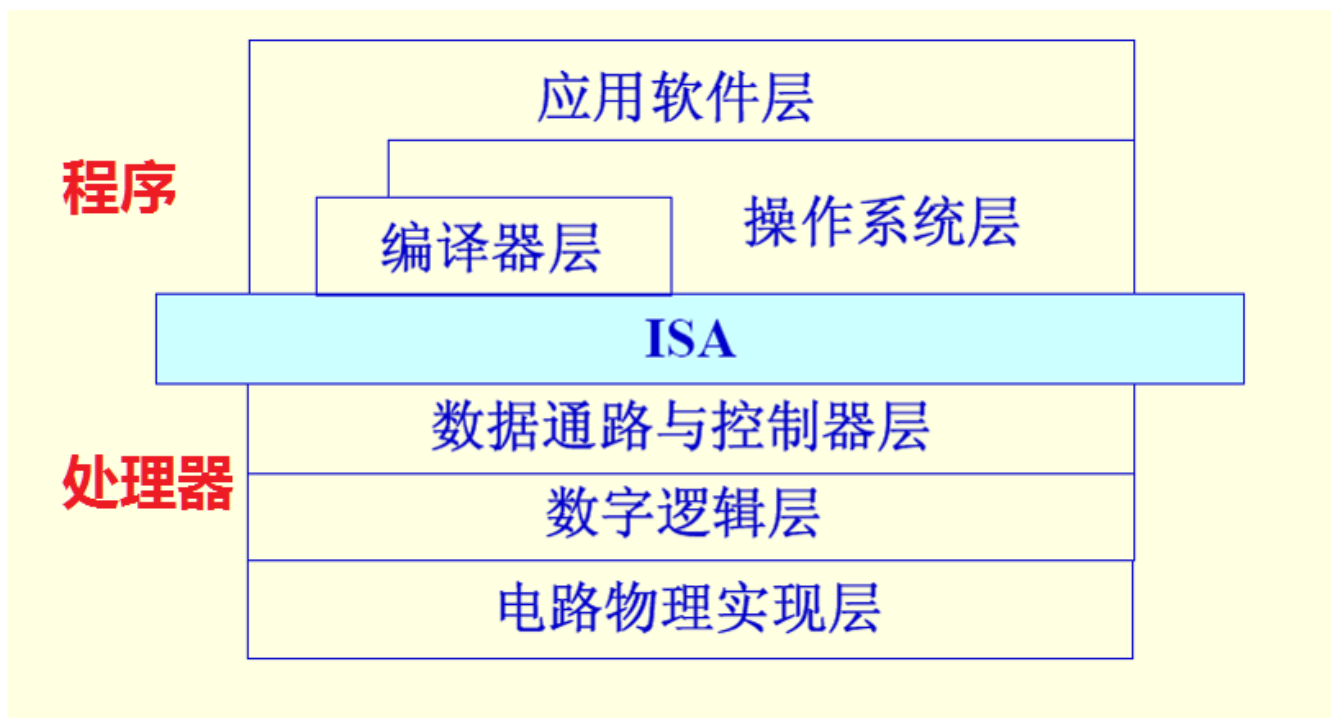


指令集

- **指令集**就是用来控制计算机系统的一套命令的集合
- **指令集是有限集合**
- 指令集中的每条指令都可以用电路实现
- 每一种处理器在设计时就规定了一系列与硬件电路相配合的指令系统
- 指令集的先进与否，也关系到处理器的性能发挥，它也是处理器性能体现的一个重要标志

指令集体系结构

- 与其它电路系统不同的是，计算机本质上是指令处理器，**指令集体系结构(ISA)**是计算机的灵魂，是计算机硬件和软件的接口





指令集体系结构

- **ISA(Instruction Set Architecture)**是计算机硬件与底层软件之间的接口，它是程序员为使一个二进制机器语言程序正确运行所需要了解的属性



我们需要哪些指令

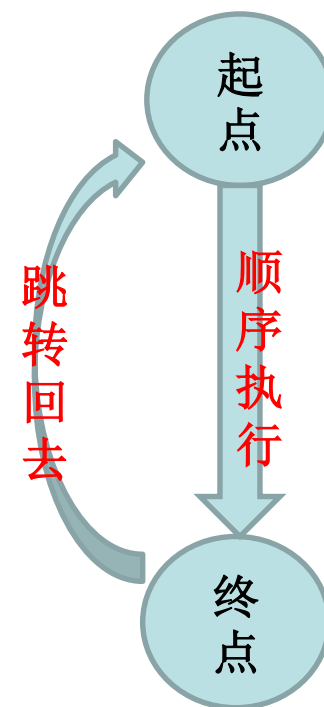
$$y(n) = \sum_{i=0}^{n-1} coef_i \bullet x(n-i)$$

- $x(n)$ 为输入信号， $y(n)$ 为输出信号， $coef_i$ 为滤波器系数
- 处理器需要有加法、乘法指令——**算术逻辑运算指令**
- **算术逻辑运算指令**还包括逻辑运算、移位运算等

我们需要哪些指令

$$y(n) = \sum_{i=0}^{n-1} coef_i \bullet x(n-i)$$

- 除了进行计算，处理器还要能实现循环，如上述计算循环了n次
- 跳转指令可以实现循环
- 循环需要在一定的条件跳出，否则就成了死循环，条件分支指令可以实现该功能
- 跳转、分支以及子程序调用与返回等指令统称**流程控制指令**





我们需要哪些指令

$$y(n) = \sum_{i=0}^{n-1} coef_i \bullet x(n-i)$$

- 现代**RISC**处理器要求算术逻辑运算的操作数只能在寄存器中，因此运算前数据要由存储器装入(**load**)寄存器中，运算完成后，再由寄存器存储(**store**)到存储器中
- 实现存储器和寄存器间数据交换的指令称为**数据传送指令**



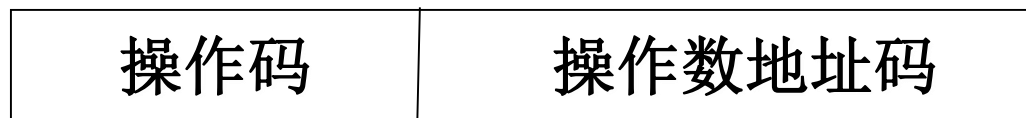
我们需要哪些指令

- 有了**算术逻辑运算指令、数据传送指令、流程控制指令这3类指令**，处理器就能够完成各种复杂的计算

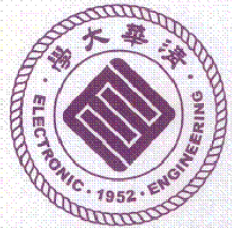


指令格式

- 指令以编码的形式如同数据一样存放在存储器中。一条指令一般由操作码和操作数地址码两部分组成

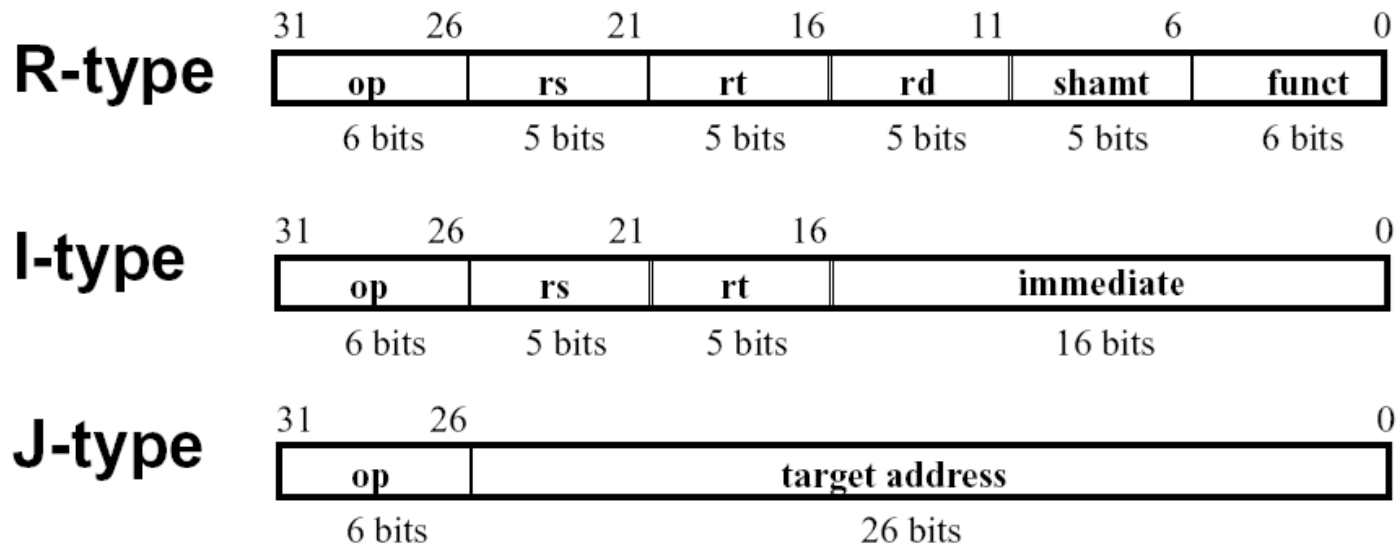


- 这些操作码和地址码被控制器译码，形成控制系统各部分的控制信号



指令集的例子

- **MIPS指令集（三种指令）**

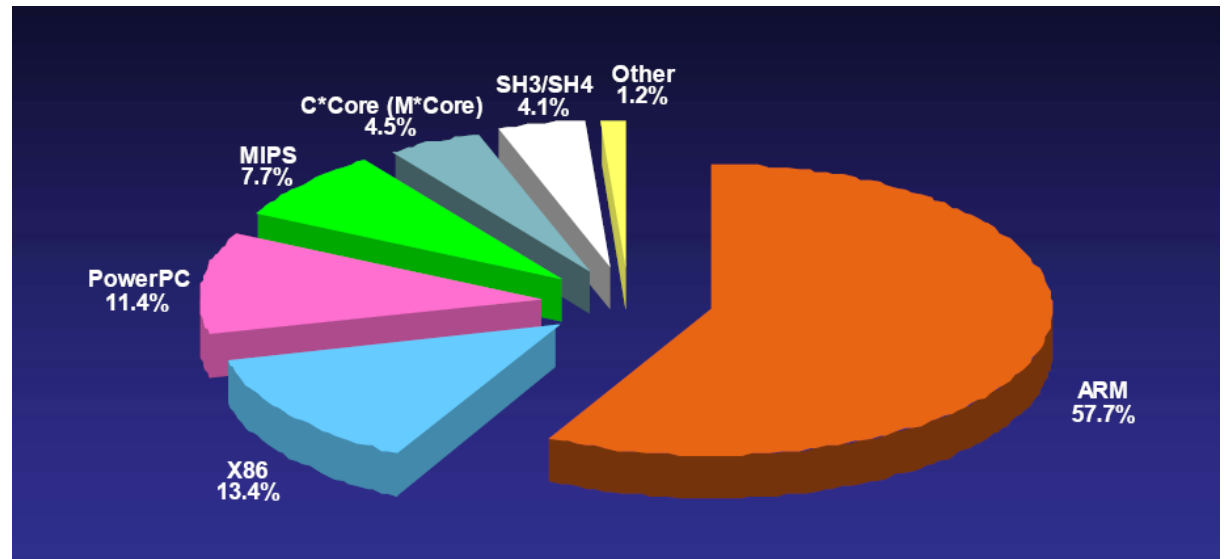


- **例子**

- **addi \$s1, \$s0, 1**
- **sw \$s1, 8000(\$s0)**

指令集体系结构的市场划分

- 虽然生产处理器的厂家很多，但世上却只有为数不多的指令集体系结构
 - ARM
 - X86
 - PowerPC
 - MIPS



ISA的地位

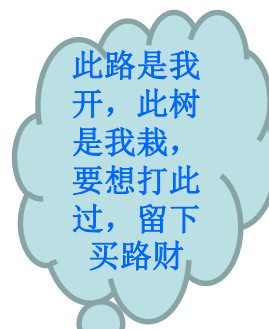
- **ISA**背后是一个生态链，而不是一两家公司
- 发明一套新的**ISA**并不难，难的是你要让别人接受你发明的**ISA**



程序

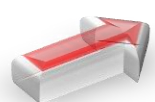
ISA

处理器





大纲

- 
- 1 从算盘到ENIAC
 - 2 通用计算机模型
 - 3 指令集体系结构
 - 4 程序和程序设计语言
 - 5 处理器的工作原理
 - 6 性能问题
 - 7 小结



什么是计算机程序？

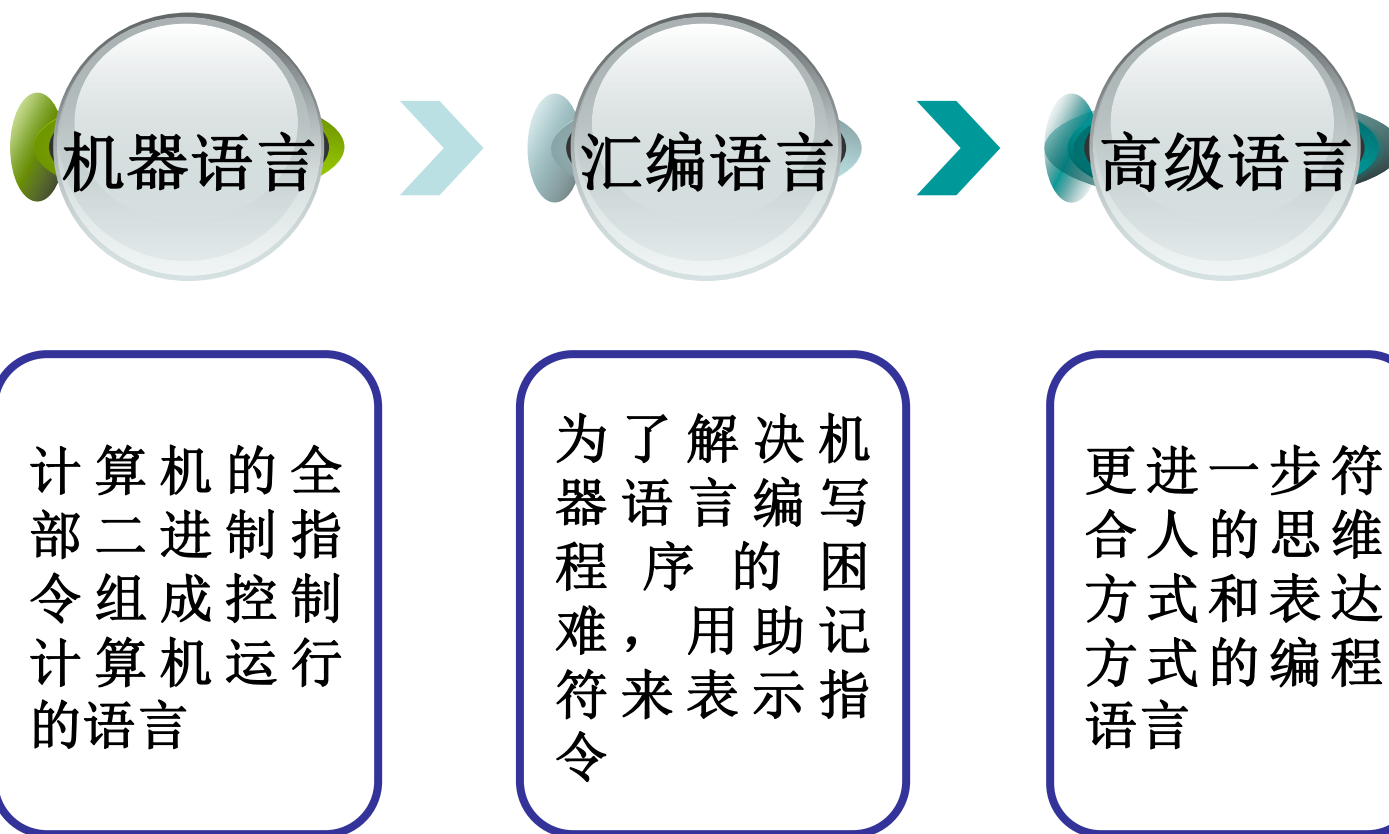
- 计算机程序就是为实现特定目标或解决特定问题而用计算机程序设计语言编写的命令序列的集合
 - 程序是由命令序列组成的，它告诉计算机如何完成一个具体的任务
 - 由于现在的计算机还不能理解人类的自然语言，所以还不能用自然语言编写计算机程序



程序设计语言

- 程序设计语言，就是一组用来定义计算机程序的语法规则
- 程序设计语言能够让程序员准确地定义计算机所需要使用的数据，并精确地定义在不同情况下所应当采取的行动
- 在过去的几十年间，发明了大量的程序设计语言：**C、C++、BASIC、FORTRAN、COBAL、Matlab、.....**

程序设计语言



机器语言

- MIPS机器语言编写的Hello, World.

```
0011110000000000100010000000000001  
0011010000100100000000000000000000  
0010010000000001000000000000000100  
00000000000000000000000000000001100  
00100100000000010000000000000001010  
00000000000000000000000000000001100
```

- 天啊，要这么编程序.....



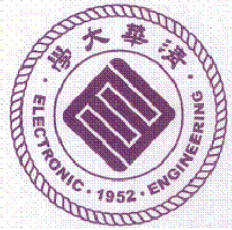


汇编语言

- 用有意义的助记符代替难以理解和记忆的指令字

```
main:      .text
           la      $a0, str
           li      $v0, 4
           syscall          # print string
           li      $v0, 10
           syscall          # exit

           .data
           .align 2
str:
           .ascii "Hello world."
```



汇编语言

汇编语言源程序

```
addi $s1, $0, 1  
addi $s1, $s1, 2  
sw $s1, 8000($0)
```

汇编器

```
00100000000010000000000000000001  
00100001000010000000000000000010  
10101100000010000001111101000000
```

机器语言程序

- 汇编语言的助记符形式的指令必须翻译成机器语言二进制指令才能在计算机上执行, 实现这种翻译的程序称为**汇编器 (assembler)**

汇编语言

- 汇编语言与机器语言是一一对应的，所以汇编语言仍然太繁琐，开发效率仍然十分低下，程序的编写太困难，所以.....





高级语言

- 更加符合人的习惯，与机器结构无关

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    printf("Hello, world!\n");
    return 0;
}
```

```
package test.hello;
```

```
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

```
using namespace std;
```

```
int main() {
    cout << "Hello, world!" << endl;
    return 0;
}
```

```
print "Hello, world!"
```



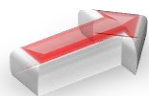

高级语言

- 不同的高级语言具有不同的语法和风格，适合于不同的应用领域
- 高级语言同样不能被计算机所直接执行，必须通过一个中间过程，可行的中间过程有编译和解释两种
 - 编译（**Compile**）：一次翻译，重复运行，效率更高
 - 解释（**Interpret**）：随执行随翻译，更灵活



大纲

- 1 从算盘到ENIAC
- 2 通用计算机模型
- 3 指令集体系结构
- 4 程序和程序设计语言
- 5 处理器的工作原理
- 6 性能问题
- 7 小结



处理器

- **处理器**是计算机的核心，它执行算术运算、逻辑判断、控制存储器访问、以及激活I/O设备等操作
- 从设计角度看，处理器包含两个主要的组成部分：
 - 数据通路
 - 控制器





数据通路

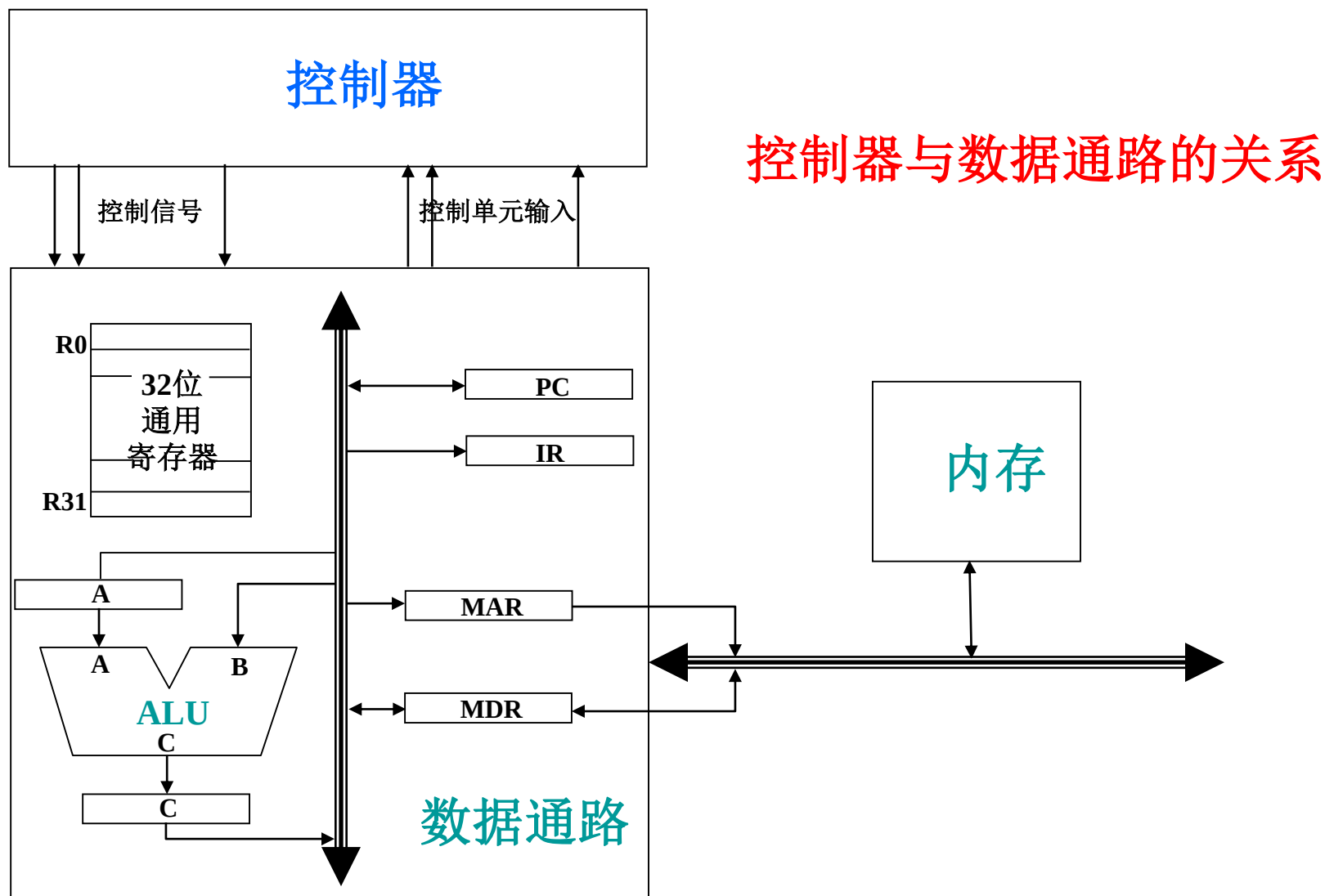
- 数据通路有两种设计方法：
 - 专用通路
 - 总线
- 专用通路是源部件和目标部件之间传送数据的一组专用信号线
- 总线(**bus**)是多个源部件和多个目标部件之间按**分时**复用的方式传送数据的一组**公用**信号线



控制器

- 提供控制信号，指挥和控制计算机所有部件协调工作，从而使计算机能够按照程序规定的指令执行次序，自动地连续执行指令

控制器与数据通路的关系





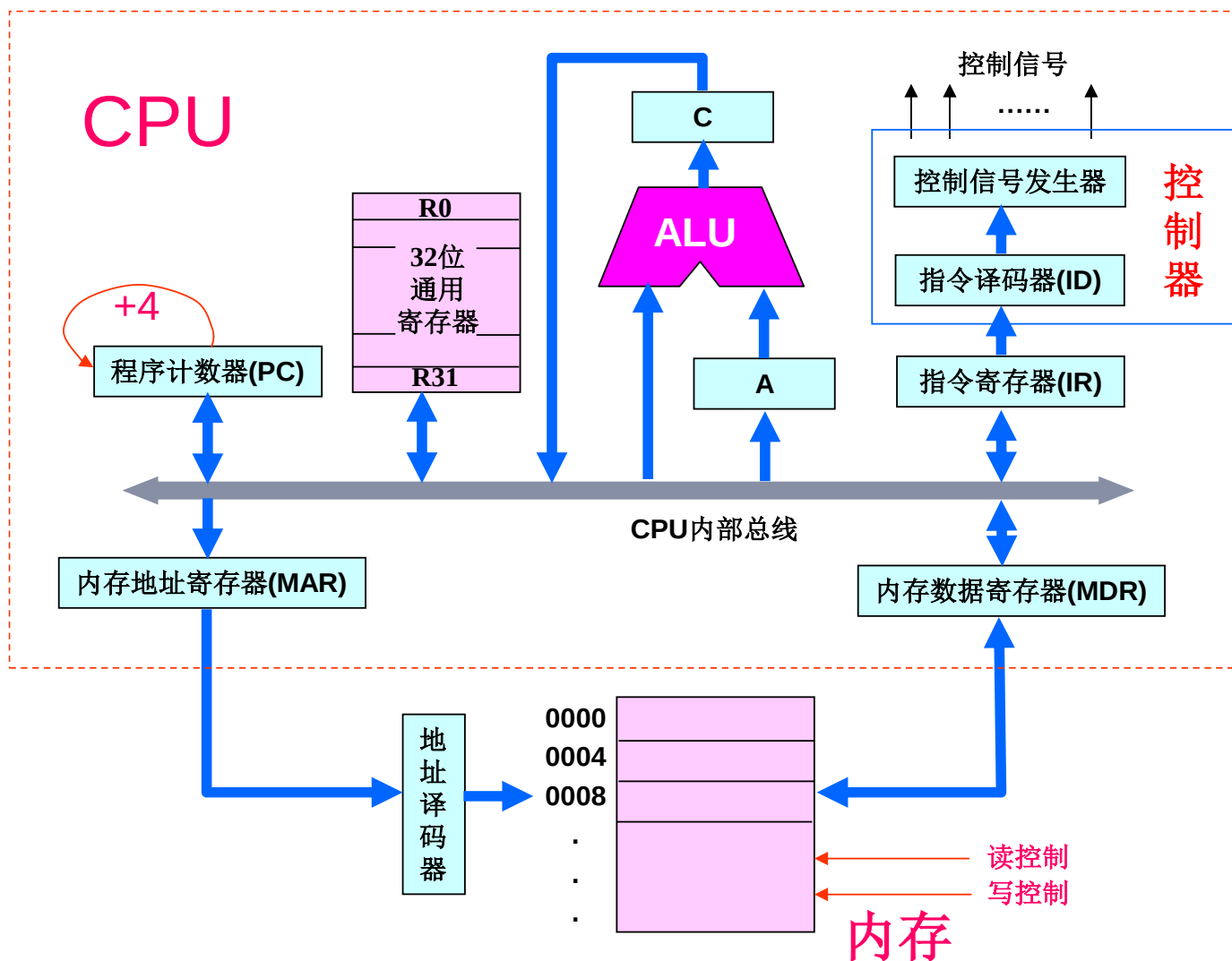
计算机怎样执行程序

问题：计算 $1+2=?$

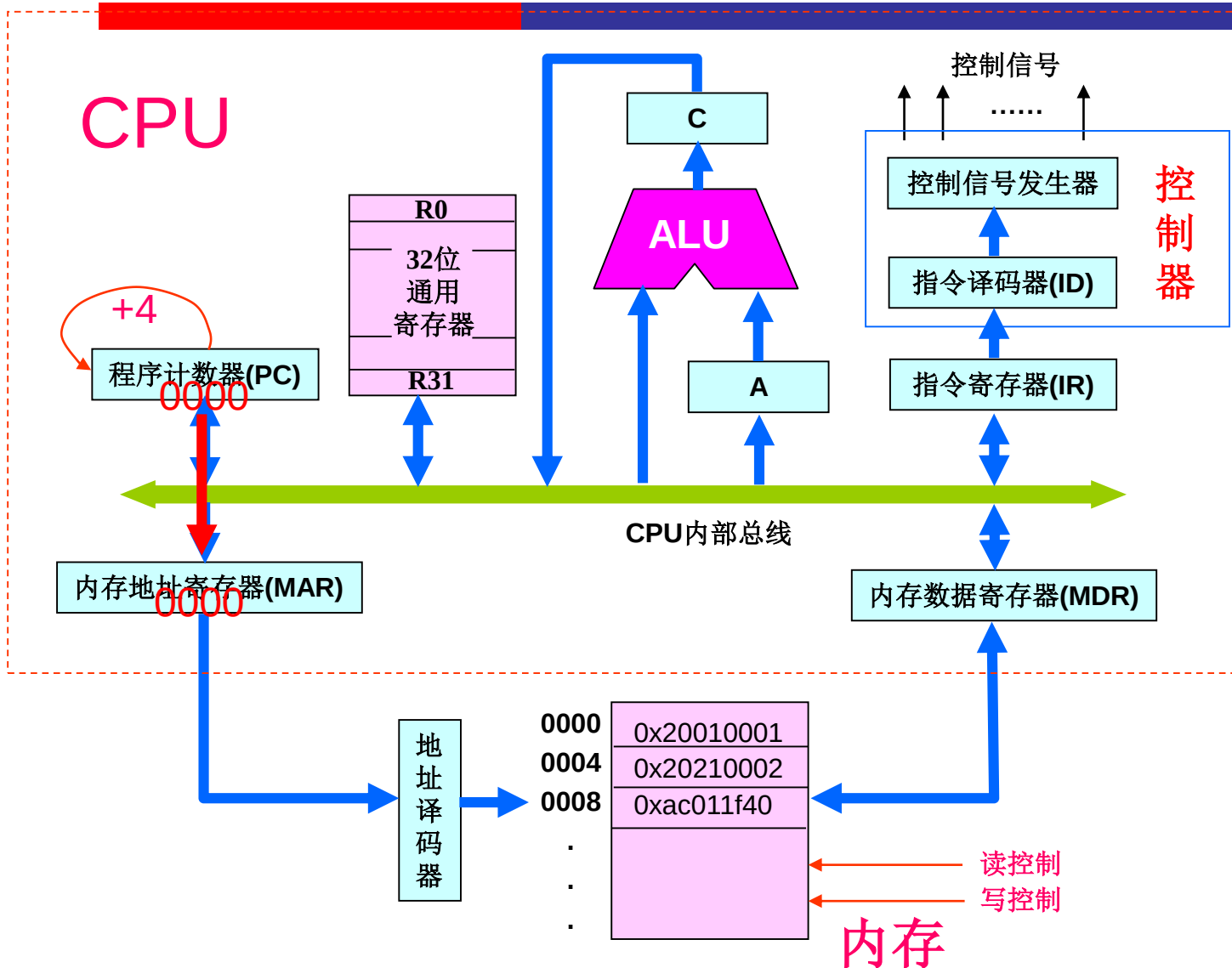
MIPS汇编语言程序

<code>addi \$1, \$0, 1</code>	<code>0x20010001</code>	$\$1=0+1=1$
<code>addi \$1, \$1, 2</code>	<code>0x20210002</code>	$\$1=\$1+2=3$
<code>sw \$1, 8000(\$0)</code>	<code>0xac011f40</code>	$\$1$ 的内容存放到地址为8000的内存单元

单总线MIPS计算机

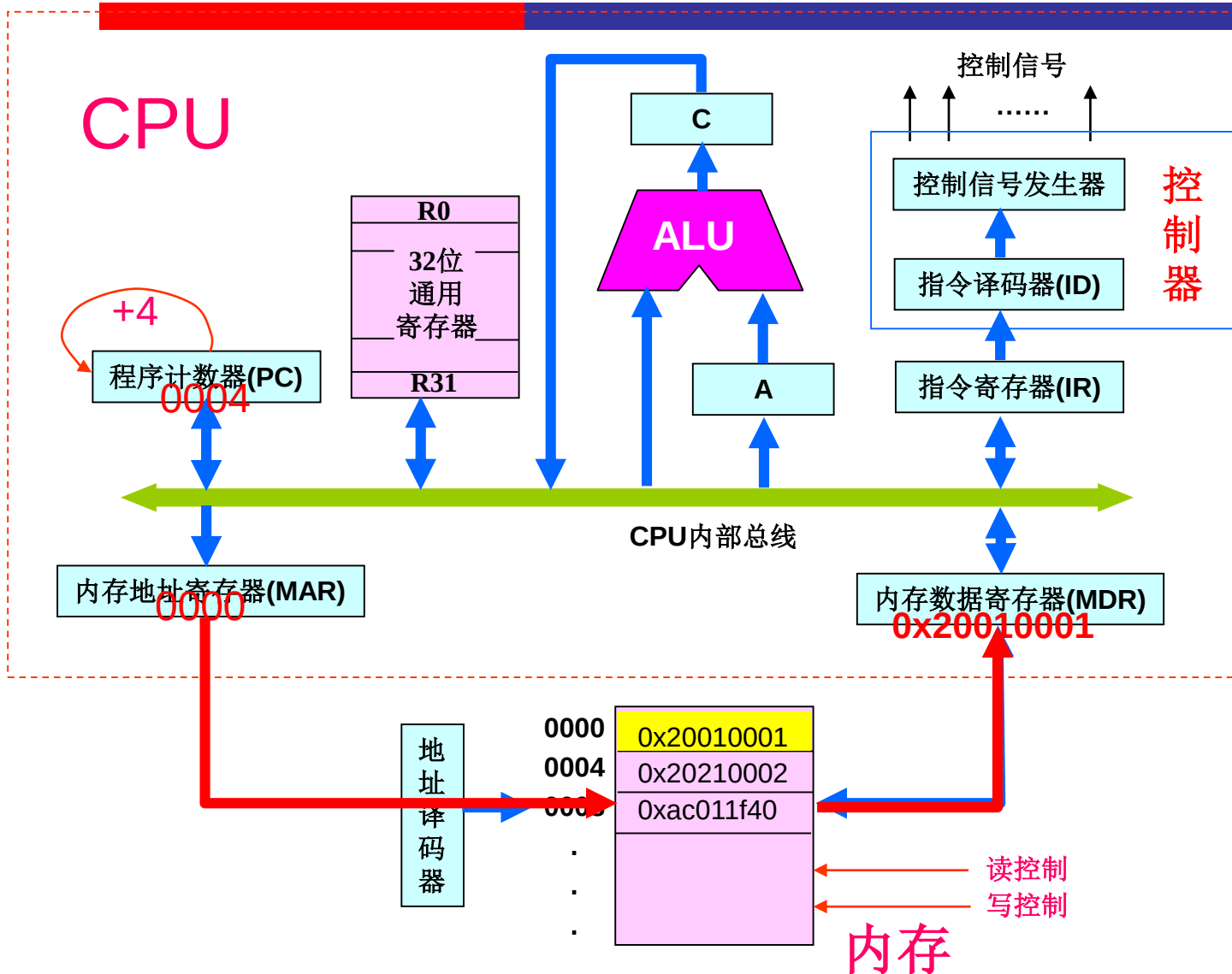


计算机怎样执行程序



addi \$1, \$0, 1
取指:
T0

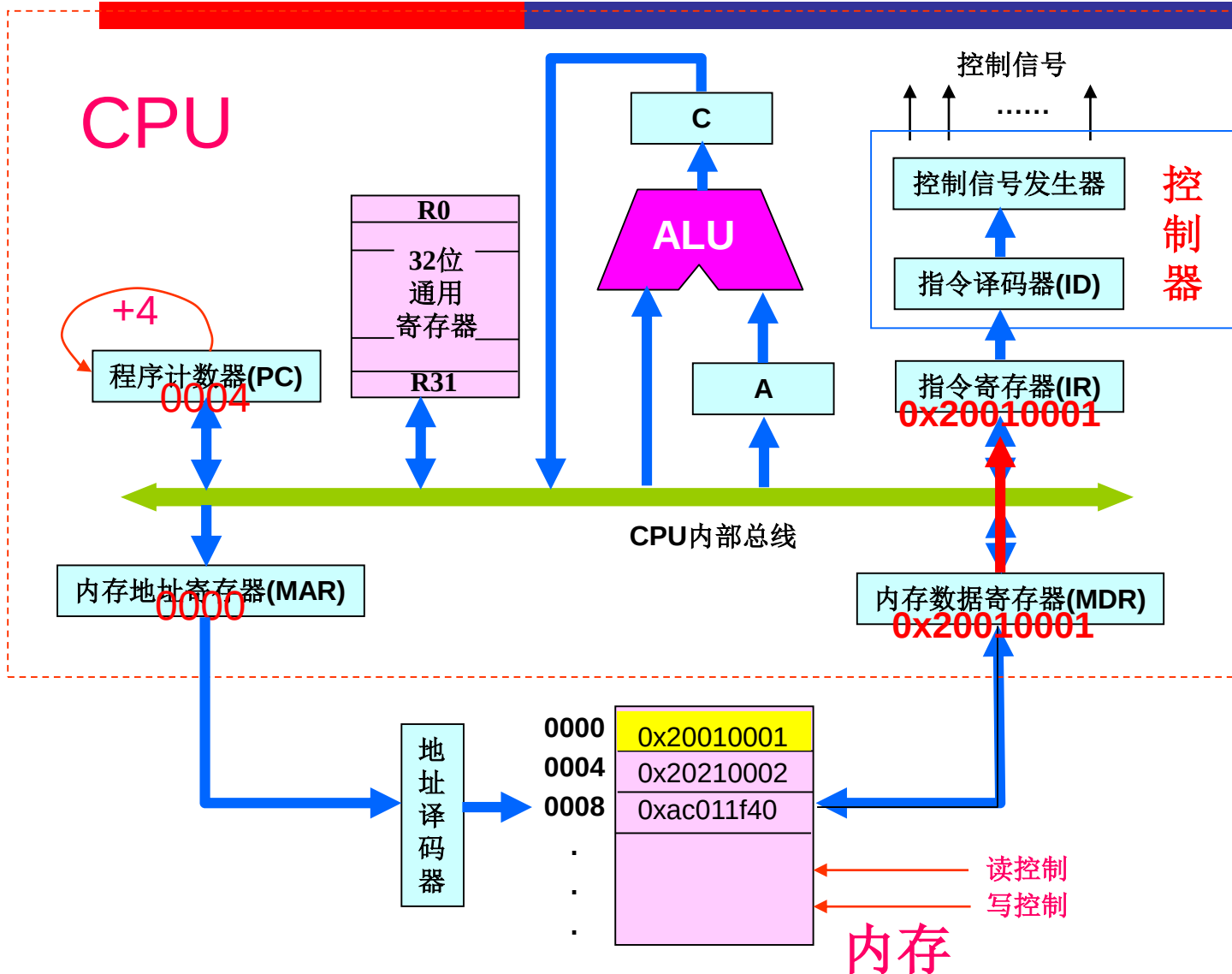
计算机怎样执行程序



addi \$1, \$0, 1
取指:
T1

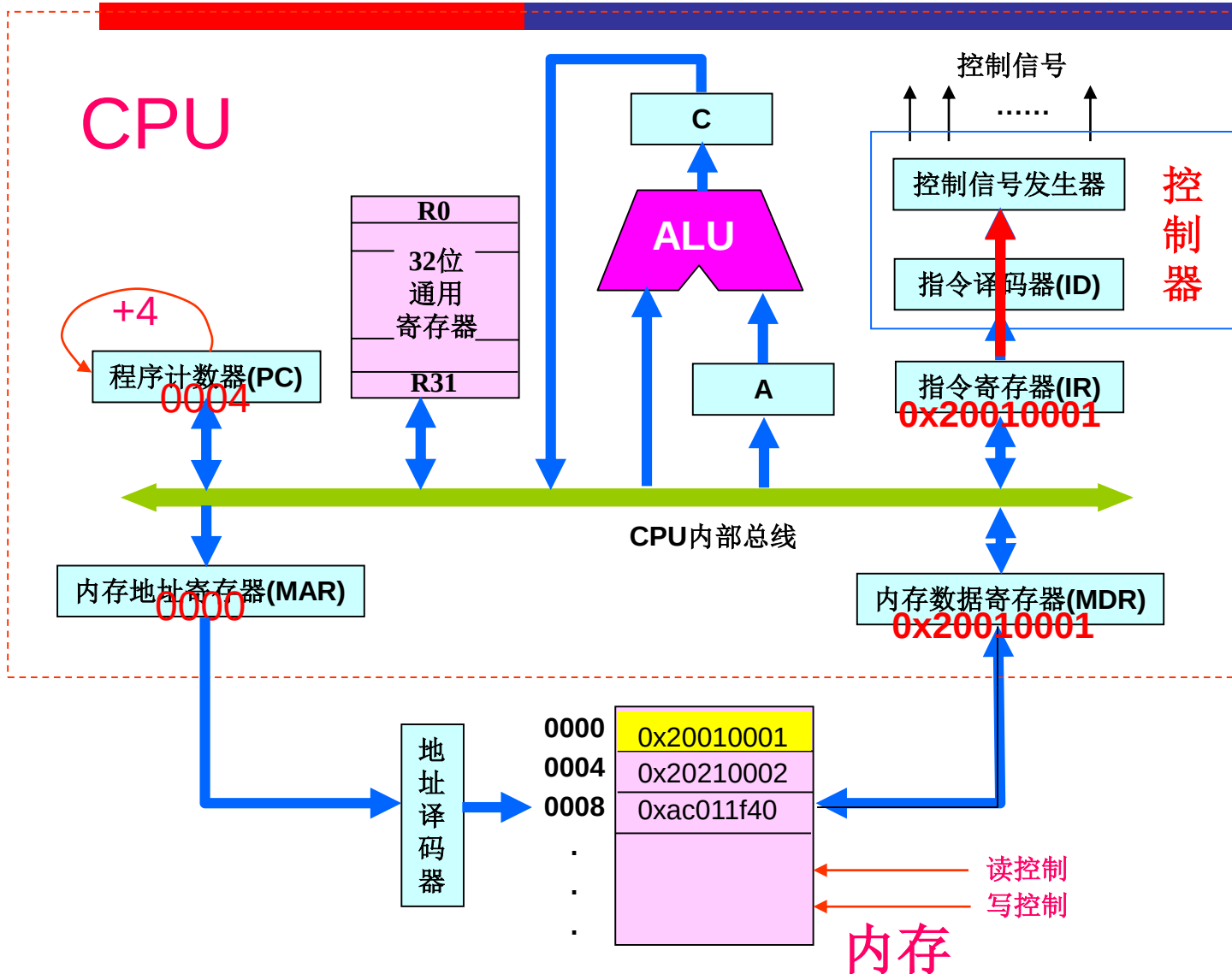


计算机怎样执行程序

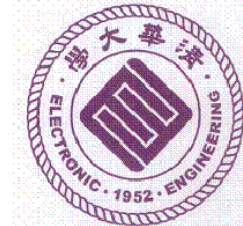


addi \$1, \$0, 1
取指:
T2

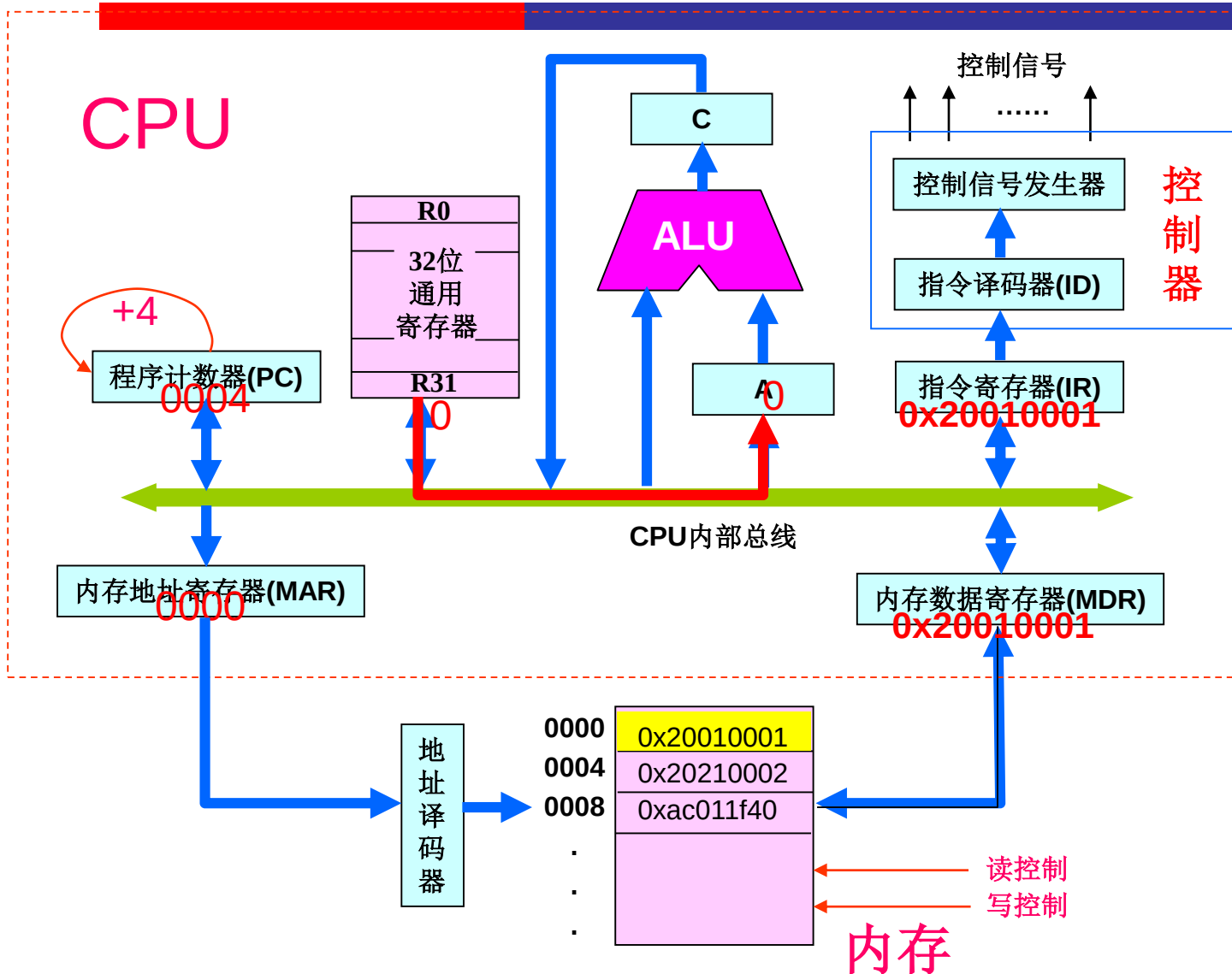
计算机怎样执行程序



addi \$1, \$0, 1
译码:
T3

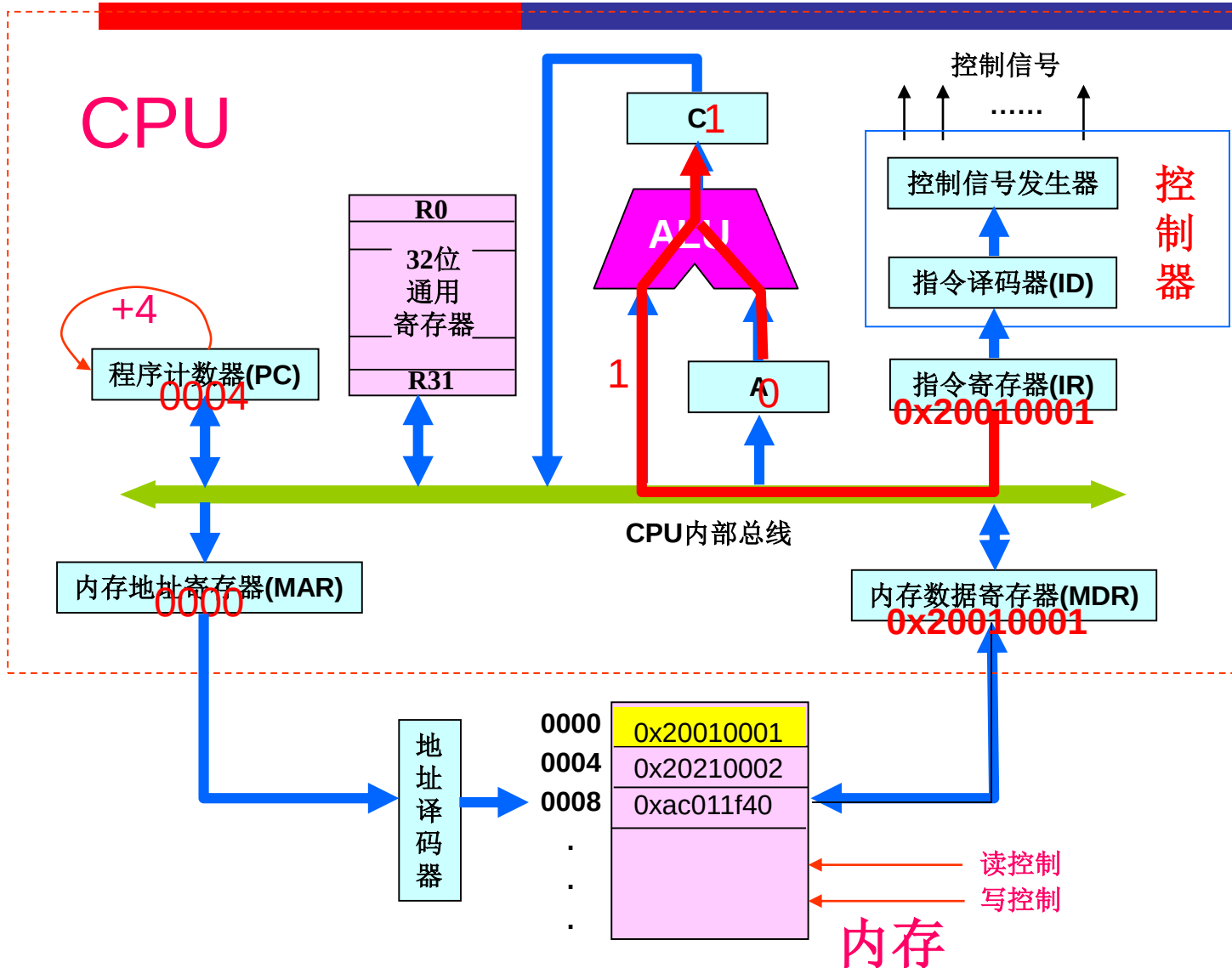


计算机怎样执行程序



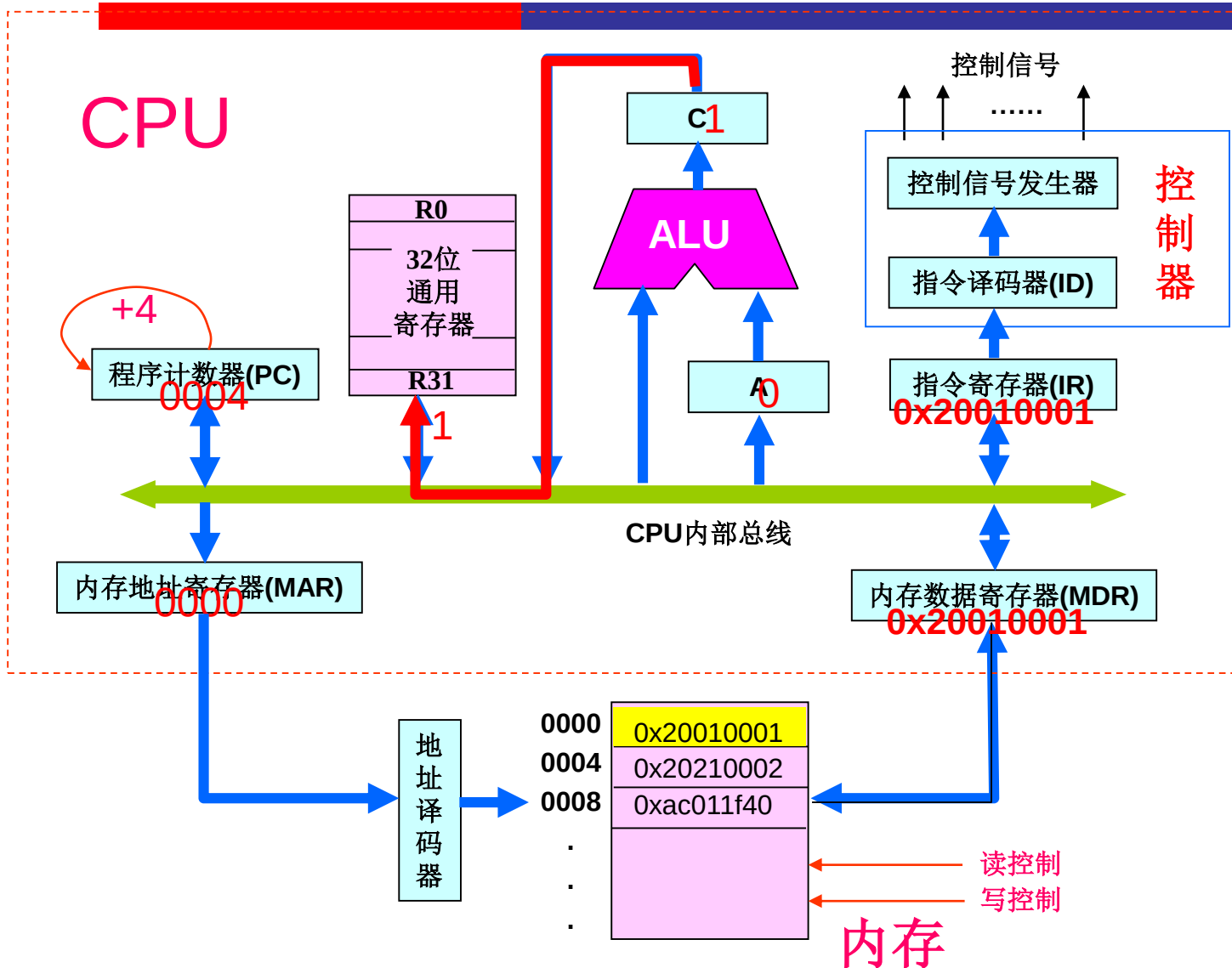
addi \$1, \$0, 1
执行:
T4

计算机怎样执行程序



addi \$1, \$0, 1
执行:
T5

计算机怎样执行程序



addi \$1, \$0, 1
执行:
T6



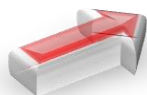
处理器

- 处理器是一个复杂的数字系统：
 - **ALU**——实现算术和逻辑运算
 - 数据通路——由处理器内部的各种连线和各种辅助性质的寄存器组成
 - 控制器——提供控制信号，指挥和控制计算机所有部件协调工作
- **ALU**一般由组合逻辑电路实现、数据通路和控制器既包含组合逻辑电路，也包含时序逻辑电路



大纲

- 1 从算盘到ENIAC
- 2 通用计算机模型
- 3 指令集体系结构
- 4 程序和程序设计语言
- 5 处理器的工作原理
- 6 性能问题
- 7 小结



更快更强

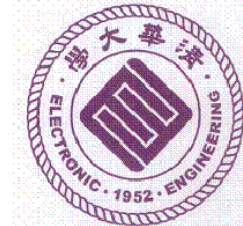
- 处理器的性能对于计算机来讲至关重要
- 让处理器变得更快更强大成为贯穿计算机发展史的重要目标
 - **ENIAC** : 5000次加法/秒
 - 神威太湖之光: 2017年Top 500中排名第1位, 它使用了中国自己的处理器(申威26010), 峰值速度达到了125435.9TFlops/s
 - 不到70年间, 10^{14} 倍的计算速度差距, 处理器的发展功不可没
- 怎样提高处理器的性能呢?





处理器性能

- 对计算机性能进行比较时，最基本的标准就是时间标准——执行同样的程序所需时间最短的计算机就是最好的计算机



处理器性能公式

执行一个
程序所花
费的时间

$P =$

$I \times CPI \times T$

程序所需
执行的总
指令数

每条指令
执行的平
均周期数

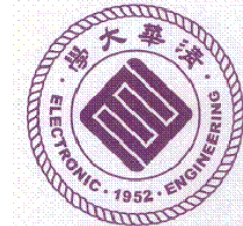
以毫秒或
纳秒计的
时钟周期



影响计算机性能的因素

$$P = I \times CPI \times T$$

影响因素	影响
算法	I、CPI
程序设计语言	I、CPI
编译器	I、CPI
ISA	I、CPI、T
硬件实现	CPI、T



提高性能的常见手段

减小I值

增强指令集中指令的能力，减小一个程序需要执行的指令数目

减小T值

采用各种技术手段提高处理器的主频，减小一个时钟周期的时间

减小CPI值

增加指令执行的并行程度，减小CPI



提高性能的常见手段

$$P = I \times \text{CPI} \times T$$

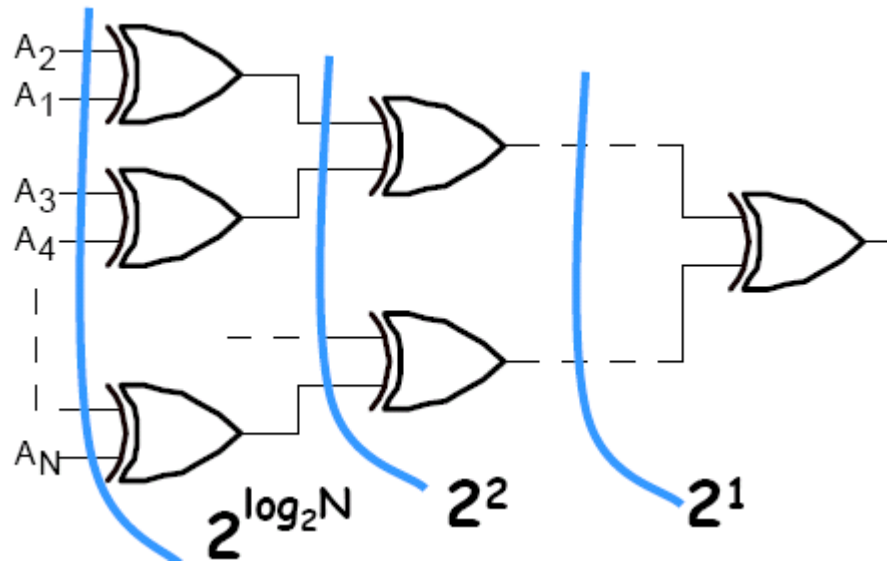
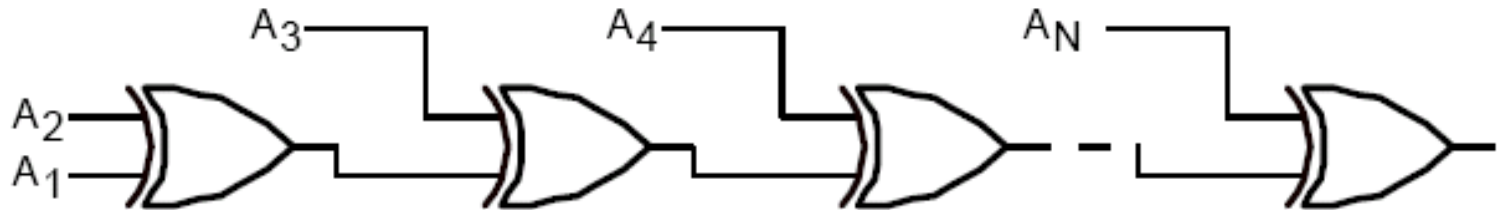
- 但是，公式中的三项并不是相互独立的，它们之间有着复杂的联系，减少三项中的任意一项都有可能增加其他两项



提高计算机性能的途径

- 一些技术可以在保持其他两项不变的前提下减少其中的一项：
 - 采用优化编译技术，在目标代码中消除冗余代码，可以减少指令数，但是并不提高**CPI**和时钟周期
 - 采用**快速电路技术**或**更为先进的结构**减少信号传输延迟，可以减少时钟周期，但是并不提高**CPI**和指令数

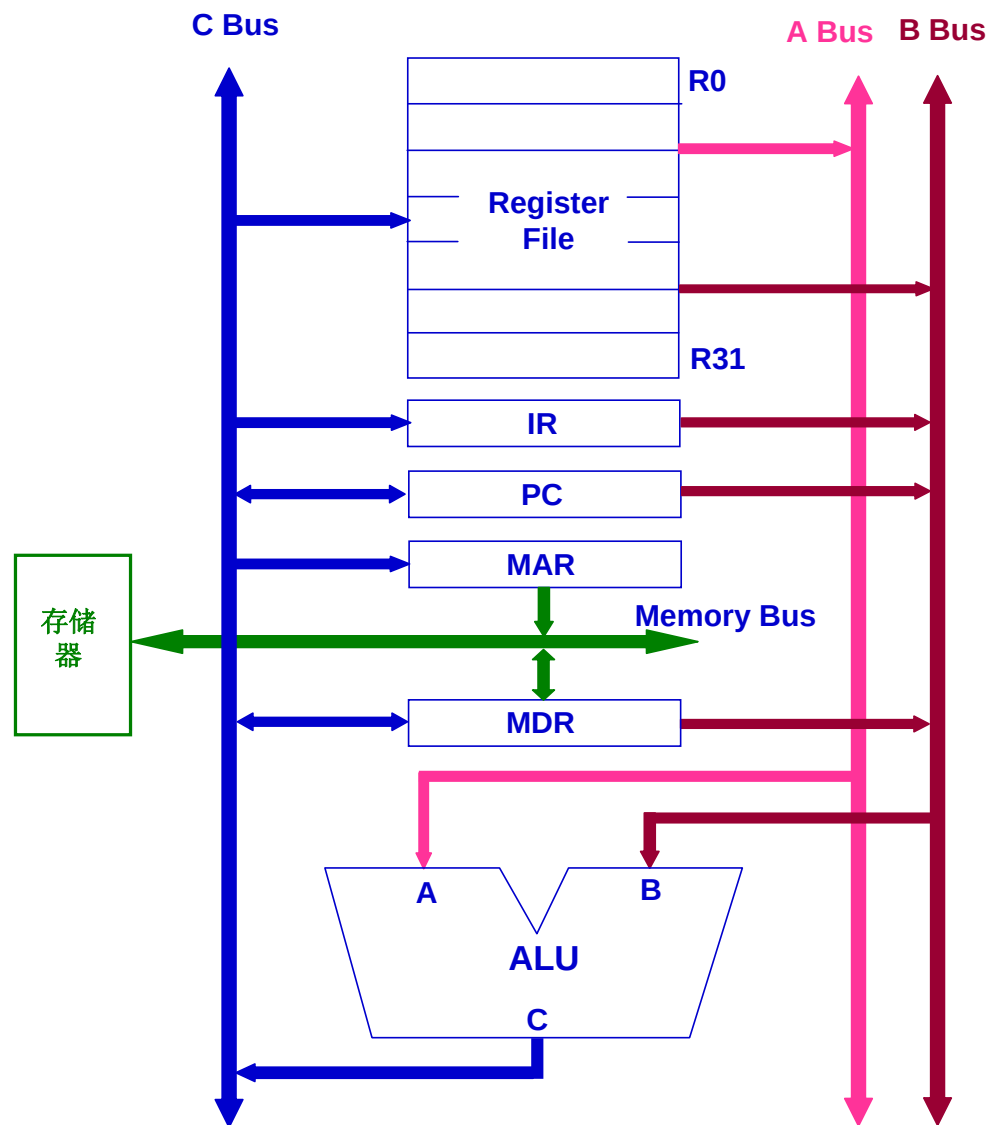
快速电路技术



新的设计速度更快
面积不变

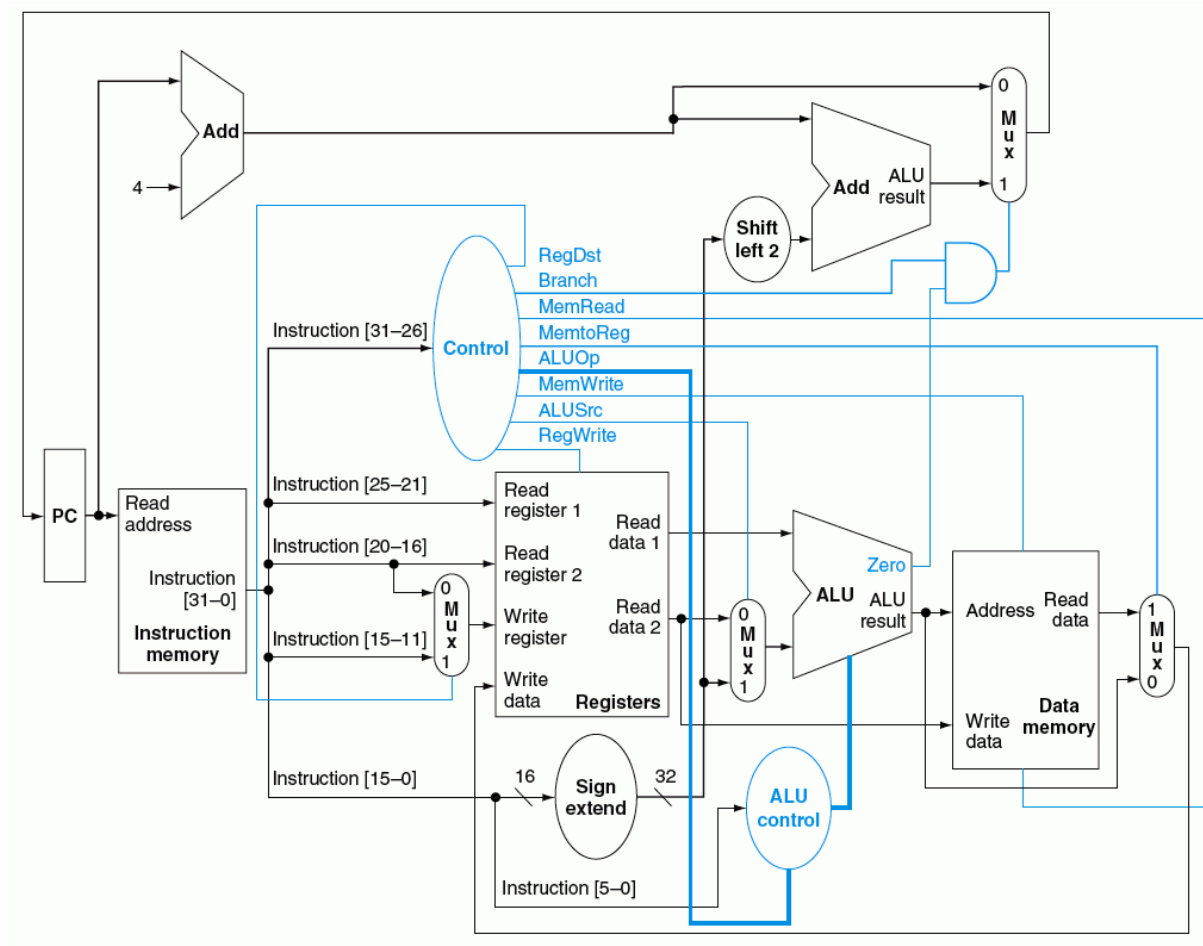
三总线结构

- 减少CPI
- 并不提高T和I



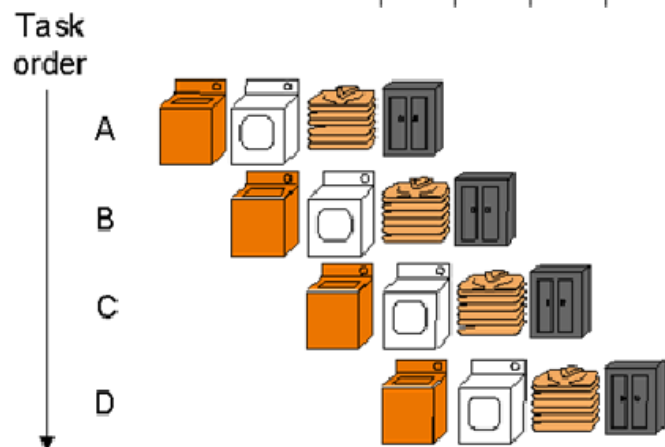
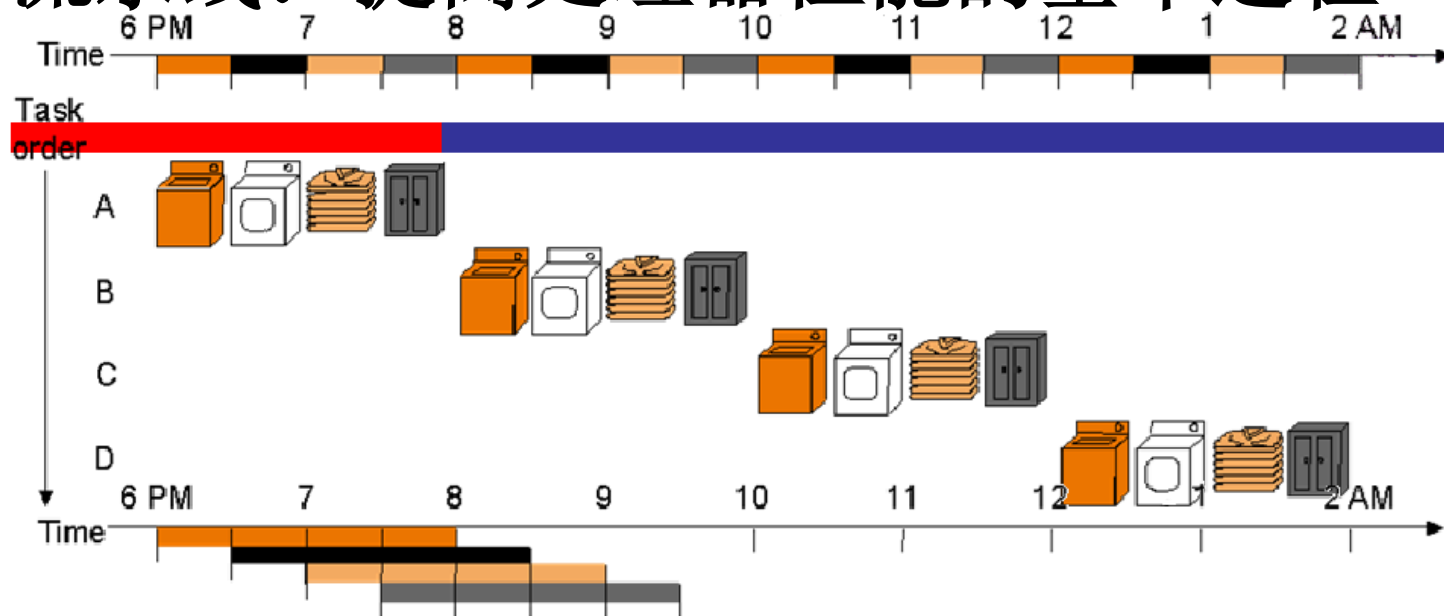
专用通路结构

- 单周期MIPS处理器
- CPI=1
- 潜在地增加T



流水线

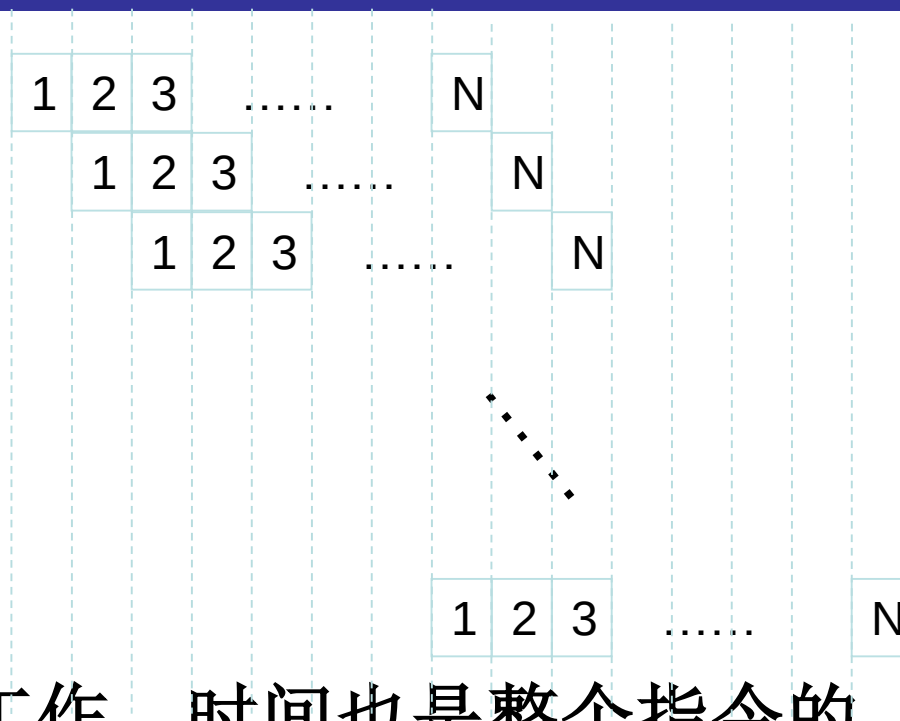
- 流水线：提高处理器性能的基本途径





流水线带来的性能改进

- **N级流水线**

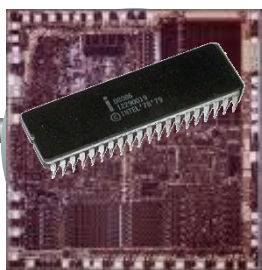


- **性能分析:**

- 每个周期做**1/N**的工作，时间也是整个指令的**1/N**
- 每个周期有一条指令完成执行
- 性能提高**N**倍.....

超流水线

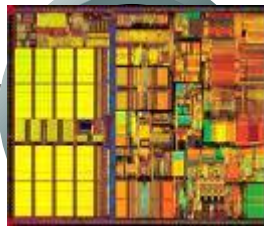
8086 → 80486 → Pentium III → **Pentium 4**



取指-执行
2级流水



取指-译码
-执行-写回
5级流水



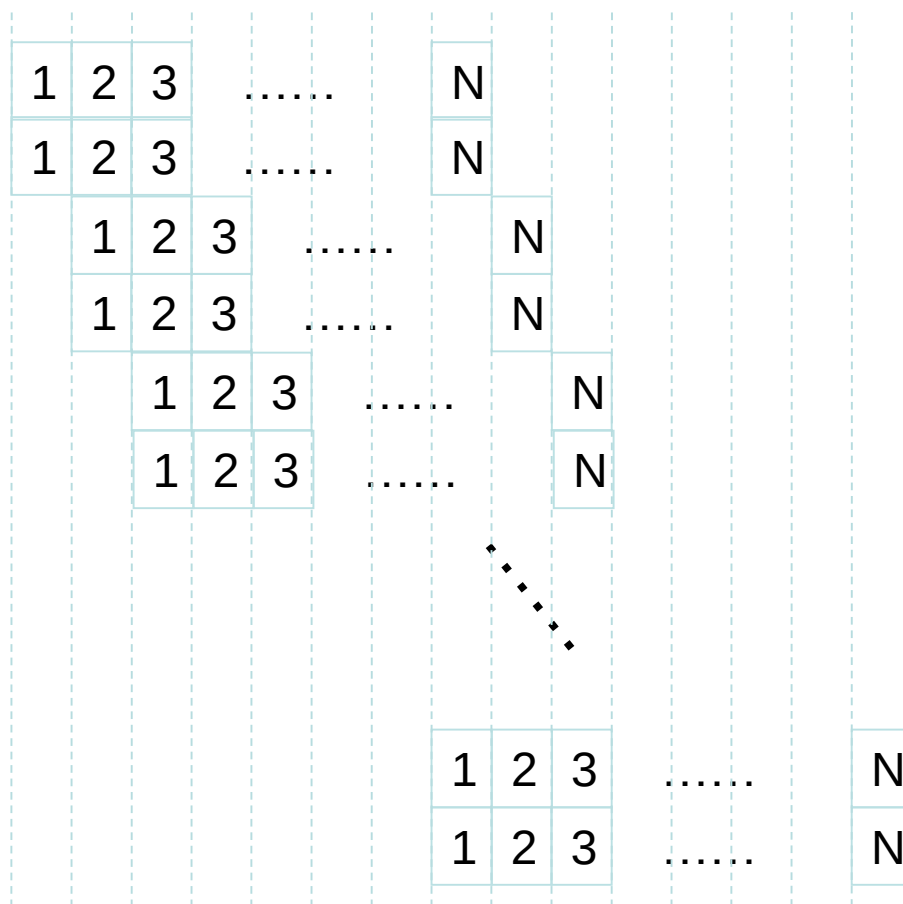
10级流水



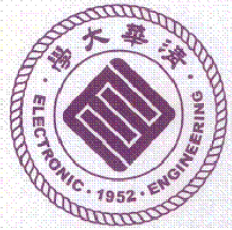
20~31级

超标量与流水线

- 一条流水线不够用，那就多条吧
- 超标量流水线结构

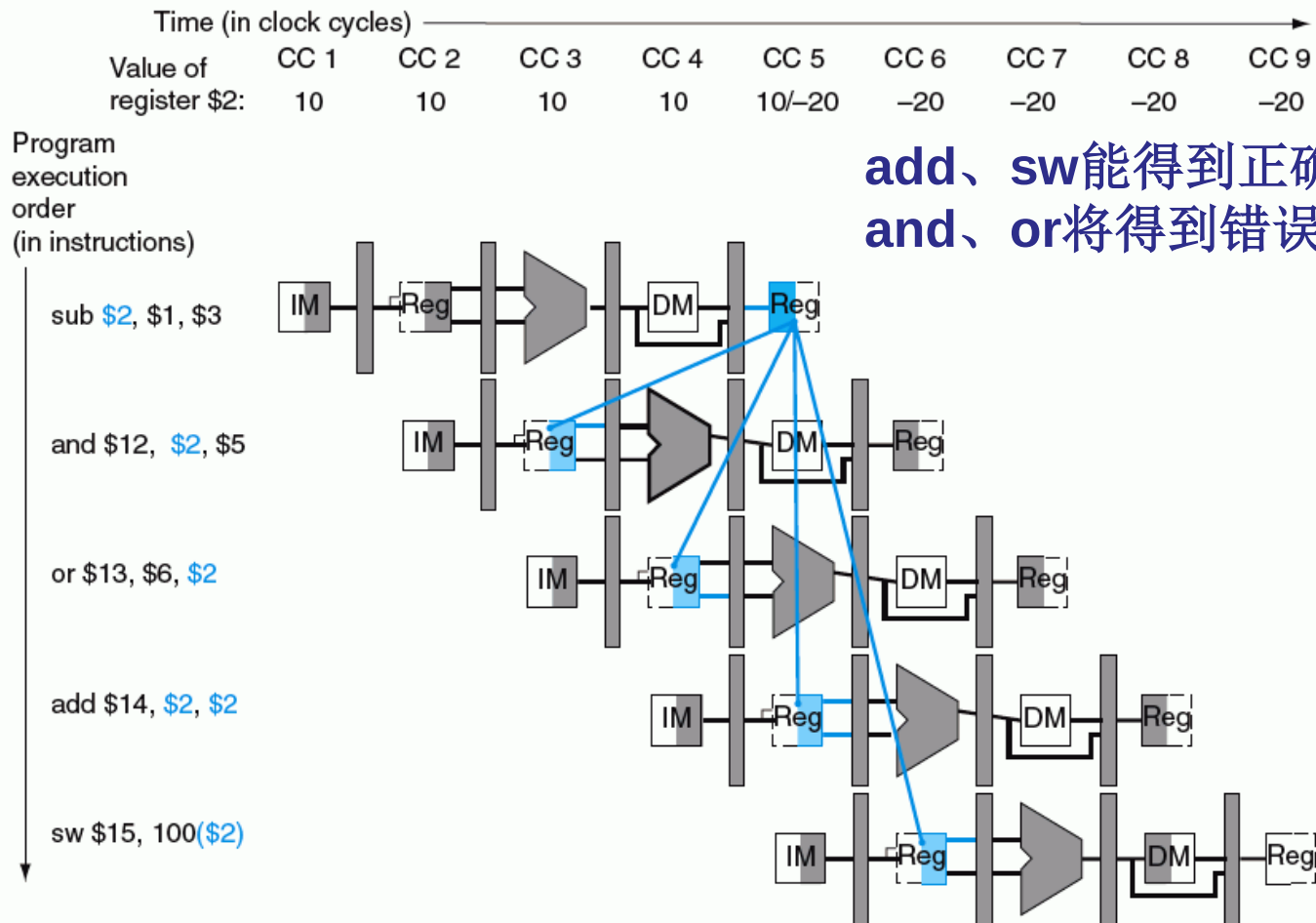


- 可是，问题在于.....



指令的相关性

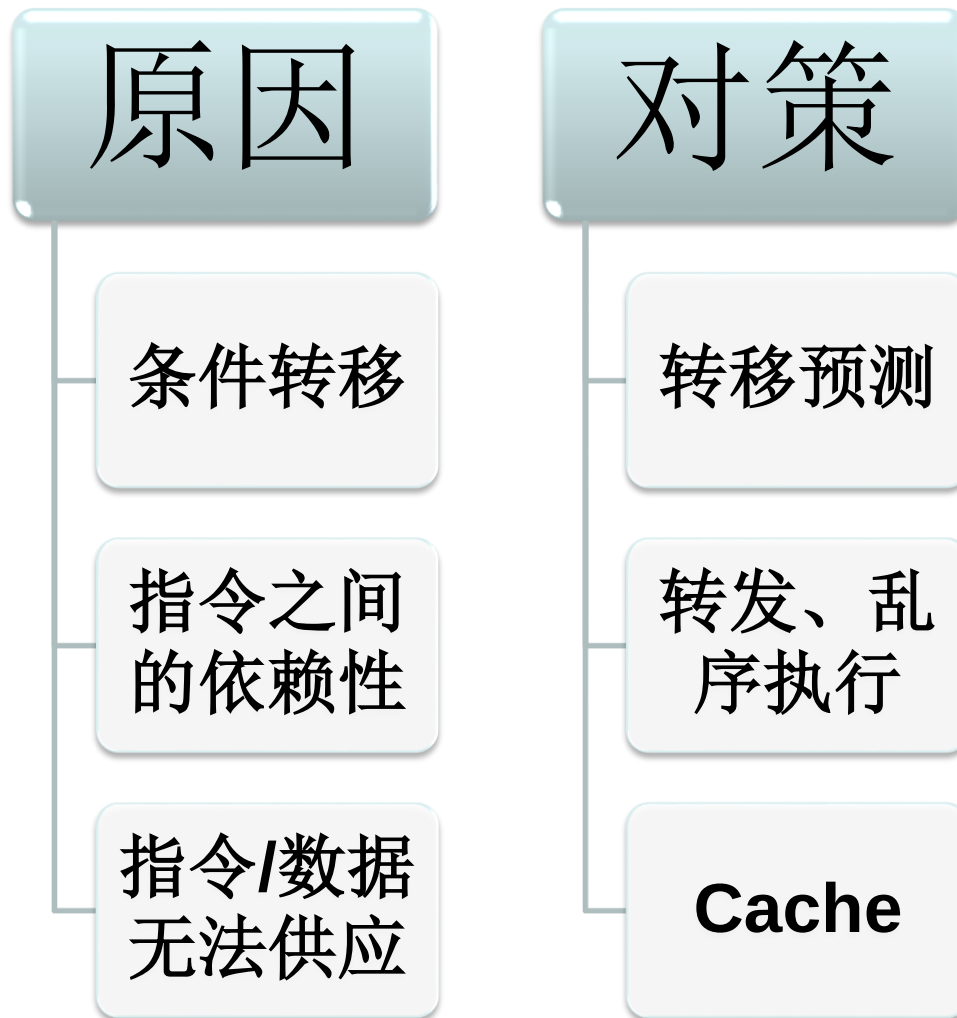
- 指令之间的相关性造成流水线中的竞争现象



add、sw能得到正确的结果
and、or将得到错误的结果



当流水线“流”不起来



转移预测

转移预测和猜测执行

- 静态预测
- 动态预测
 - “历史记录”法
- 转移预测缓冲 (BTB)
- 转移延迟

说明:

T=转移取

N=转移不取

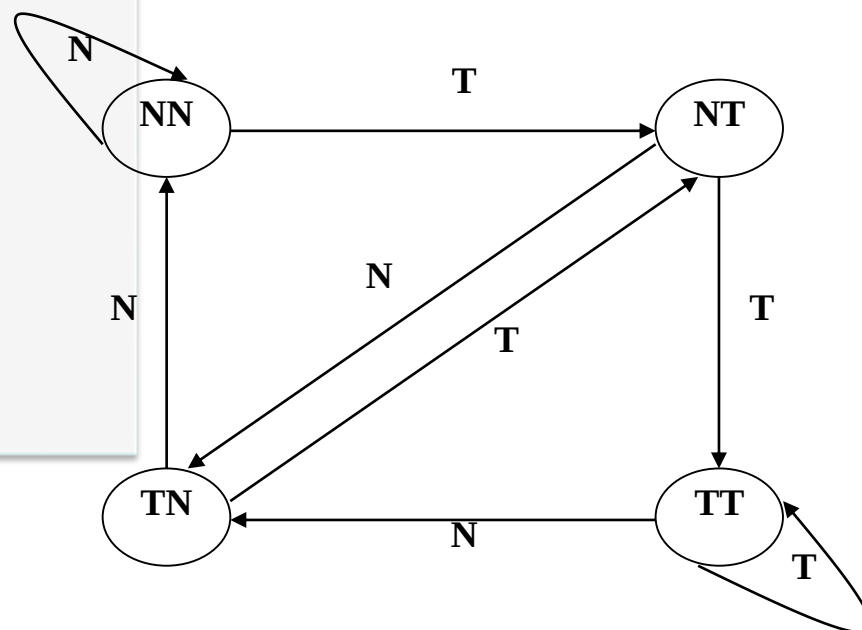
NN=前两次转移不取

NT=前两次转移不取、取

TN=前两次转移取、不取

TT=前两次转移取

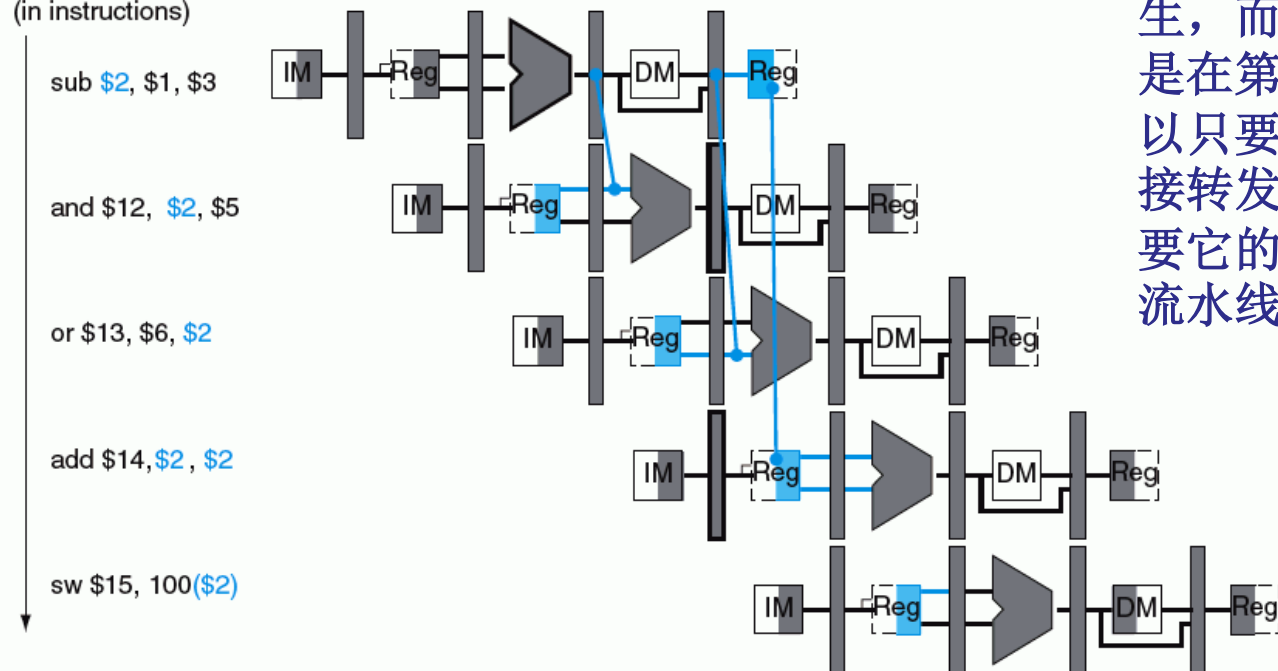
除NN预测不转移外，其它情况均预测转移。



转发

	Time (in clock cycles) →								
	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9
Value of register \$2:	10	10	10	10	10/-20	-20	-20	-20	-20
Value of EX/MEM:	X	X	X	-20	X	X	X	X	X
Value of MEM/WB:	X	X	X	X	-20	X	X	X	X

Program
execution
order
(in instructions)

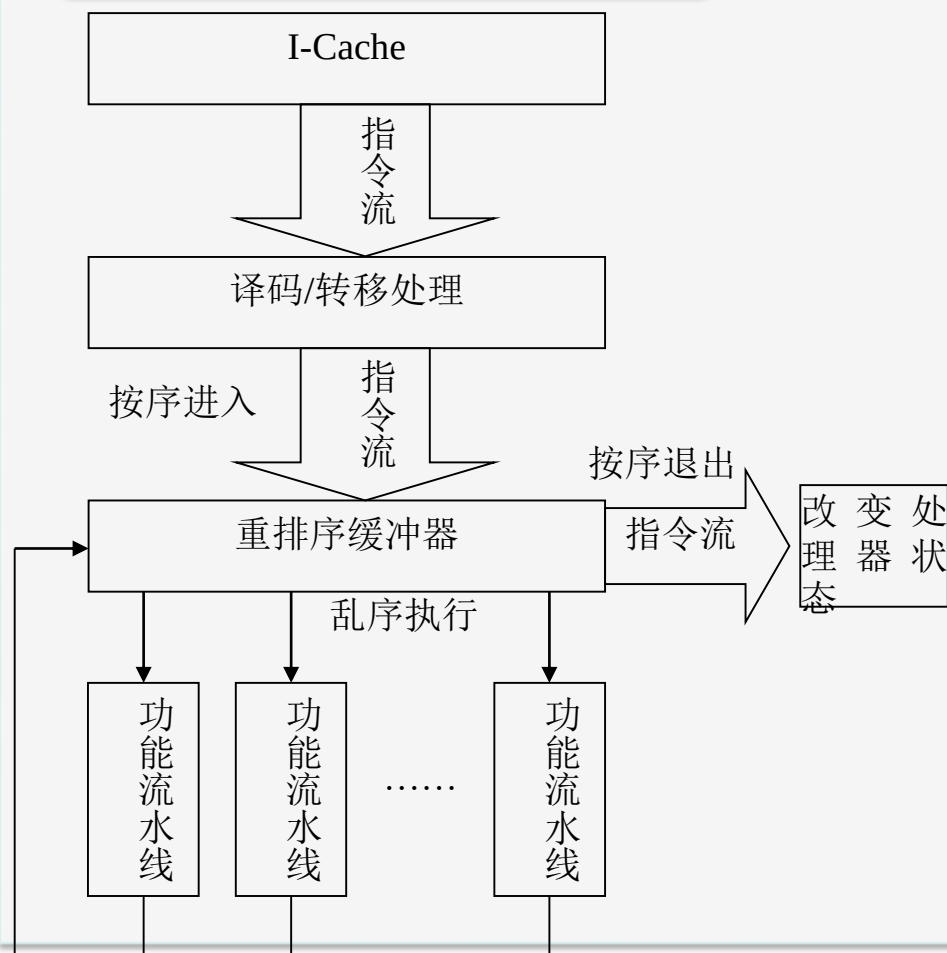


sub的结果在第3周期末产生，而**and**、**or**进入EX段是在第4和第5周期——所以只要该结果产生之后直接转发(**forwarding**)给需要它的指令，就可以避免流水线的停顿

乱序执行

- 请A、B、C三个名
人为晚会题写横幅
“春节联欢晚会”
六个大字，每人各
写两个字，怎么写？

乱序执行



存储器拖后腿了

Cache!

- 原理：局域性
- 80386：板上Cache支持；80486：片上Cache

Cache不命中时？

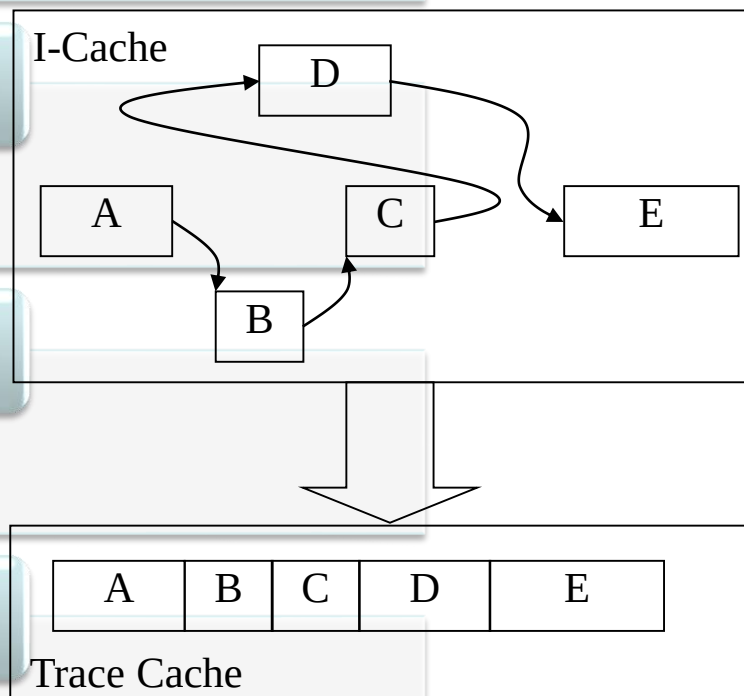
- 80486：多级Cache

Cache又成瓶颈

- Pentium：I/D-Cache

进一步提高

- Cache预取技术，Trace Cache





光Cache恐怕还不够吧

ISA

PCI

FSB

HyperTransport/QPI

总线



CPU的未来

- **Moore's Law:**

- 三种版本

- 1、集成电路芯片上所集成的电路的数目每隔**18**个月就翻一番
 - 2、微处理器的性能每隔**18**个月提高一倍，而价格下降一倍。
 - 3、用一个美元所能买到的电脑性能，每隔**18**个月翻两番。

- 从**1965**年提出至今，摩尔定律支配了微电子与计算机技术**50**余年，什么时候是摩尔定律的终结？还是会有其他的技术、其他的定律替代摩尔定律？

- 多核技术的发展带来新的挑战：

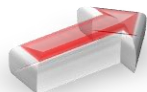
- 多少个核心够用？多核对系统中其它部分带来的挑战？
 - 多核/众核时代如何编程？

- 新的体系结构是否会带来程序模型的根本性变化？

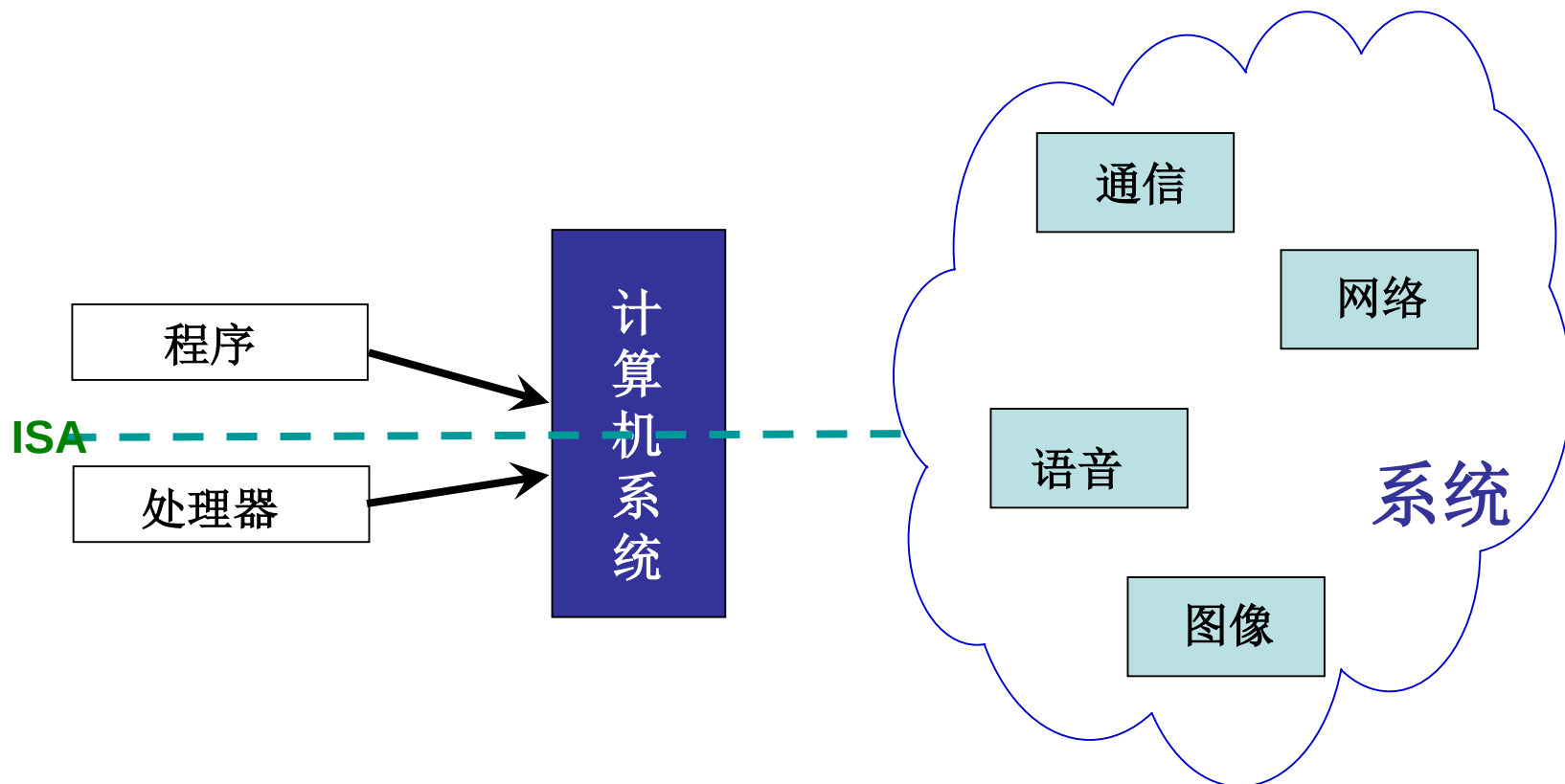


大纲

- 1 从算盘到ENIAC
- 2 通用计算机模型
- 3 指令集体系结构
- 4 程序和程序设计语言
- 5 处理器的工作原理
- 6 性能问题
- 7 小结



小结





与课程的关系

- 核心课
 - 程序设计语言
 - 数字逻辑与处理器基础
- 限选课
 - 数字系统设计（偏重处理器+IO）
 - 计算机体系结构（偏重多核）



谢谢！